Machine Management System
Version 2.0 — Technical Handbook

Tinkerers' Lab
Indian Institute of Technology Indore

Document type:	Technical Reference Handbook
Version:	2.0.0
Date:	June 2026
Status:	Released — Production
Audience:	Lab Coordinators, Developers, Maintainers

This document is the primary technical reference for MMS V2.
It covers hardware, firmware, backend, deployment, and maintenance.

Maintained by the Tinkerers' Lab technical team.
File issues and updates in the project repository.

Revision History

Rev	Date	Author	Description
1.0	Jan 2024	Original team	V1 system documentation
2.0	Jun 2026	Lab tech team	V2 complete rewrite — new RFID schema, MQTT bridge, Firebase backend

Abstract

The Machine Management System Version 2 (MMS V2) is an IoT-based access control and usage tracking platform for the Tinkerers' Lab at IIT Indore. It controls access to five workshop machines (3D printer, laser cutter, grinder, two soldering stations) using RFID cards and logs all usage to a cloud database.

MMS V2 replaces a non-functional V1 system. The V1 system had a critical RFID block address mismatch, unhashed PIN storage, non-unique MQTT client IDs, no watchdog, no cloud logging, and was never tested end-to-end.

V2 is built on the ESP32 microcontroller, uses MIFARE Classic 1K cards with a custom cryptographic sector key scheme, routes session events through a Raspberry Pi MQTT broker to Firebase, and exposes a React web dashboard for real-time monitoring and administration.

This handbook serves as the complete reference for anyone deploying, maintaining, or extending MMS V2. It is written for longevity — the assumption is that the original developers may no longer be available, and a future maintainer with general embedded systems knowledge should be able to understand and operate the system from this document alone.

Contents

List of Tables

List of Figures

Chapter 1

Project Overview

1.1 What Is MMS V2?

The Machine Management System Version 2 (MMS V2) is an embedded IoT platform for controlling and monitoring access to workshop equipment in the Tinkerers' Lab at IIT Indore. The system enforces that only authorised students can operate specific machines, logs all usage to a cloud database, and provides administrators with real-time monitoring and remote control capabilities via a web dashboard.

The system is deployed on a local area network within the lab and communicates with Firebase (Google Cloud) for permanent data storage and the administrator web interface.

1.2 Scope and Objectives

1.2.1 What MMS V2 Does

- **Access control**: RFID card authentication at each machine. A card is accepted only if (a) it is a valid MMS V2 card, (b) it is not revoked, and (c) its machine permission bitmask authorises the specific machine.
- **Machine control**: An ESP32 node controls a relay that interrupts mains power to the machine. Access granted → relay ON → machine receives power. Session ended → relay OFF → machine loses power.
- **Session tracking**: Every session (start time, end time, duration, end reason, student roll number, machine) is logged to Firebase Firestore.
- **Offline operation**: Access decisions are made locally on the ESP32, without any network call. The system functions even if the Raspberry Pi or Firebase is unreachable.
- **Remote administration**: Admins can view real-time machine status, stop active sessions, enable or disable machines, revoke lost cards, and trigger firmware updates from a web browser.
- **Card management**: A kiosk station (Flask web app + MFRC522 reader) issues personalised RFID cards to students. Each card is cryptographically personalised with the student's roll number, PIN hash, and machine permissions.

1.2.2 What MMS V2 Does Not Do

- PIN verification at the machine (Phase 4 — requires rotary encoder user interface, not yet deployed)

- Machine booking or reservation (separate future system)
- Integration with the institute student database (cards are issued manually)
- Control of AC 240V systems beyond the relay/SSR interface (wiring to machines is the lab's responsibility)
- Multi-factor authentication (planned future upgrade)

1.3 System Scale

Table 1.1: MMS V2 System Scale Parameters

Parameter	Value
Maximum machines supported	32 (per permission bitmask)
Currently deployed	5 machines
Maximum concurrent users	32 (one per machine)
Maximum registered users	Unlimited (Firestore)
Expected user base	300 students
RFID card technology	MIFARE Classic 1K
Card capacity per institution	Effectively unlimited
Offline session buffer per node	~500 sessions (50KB LittleFS)
OTA rollout time (all 5 nodes)	~5 minutes
Session data retention	Indefinite (Firestore Spark tier: 1GB free)

1.4 Deployed Machines

Table 1.2: Canonical Machine Registry

ID	Machine Name	Session Limit	Permission Bit	Zone
1	Creality 3D Printer	60 min	Bit 0	Zone A
2	Fracktal Laser Cutter	30 min	Bit 1	Zone B
3	Soldering Station 1	15 min	Bit 2	Zone C
4	Soldering Station 2	15 min	Bit 3	Zone C
5	Bosche Grinder	20 min	Bit 4	Zone D

WARNING — Three-places-in-sync rule: Machine IDs must match exactly in `config.h` (ESP32 firmware), `scripts/seed_machines.py` (Firestore seeder), and `kiosk_station/config.py` (kiosk permission UI). An ID mismatch means sessions are logged under the wrong machine or cards grant wrong machine access. See Chapter 8 and Appendix E.

1.5 Key Stakeholders

Lab Coordinator Responsible for day-to-day operations: issuing cards, managing user accounts, monitoring dashboard, handling card loss/replacement.

Admin Web app admin role: all coordinator functions plus remote stop, machine enable/disable, session log export.

Super Admin Highest privilege: OTA firmware updates, adding/removing admins, Firebase console access.

Student Holds an RFID card. No web app access. Operates machines by tapping card.

Future Maintainer Anyone who maintains or extends the system after the original developers. This document is written primarily for them.

1.6 Glossary

Term	Definition
RFID	Radio Frequency Identification. Contactless card technology.
MIFARE Classic 1K	Type of RFID card with 1024 bytes of data storage in 16 sectors.
ESP32	Dual-core 240MHz microcontroller with built-in WiFi and Bluetooth, by Espressif.
MQTT	Message Queuing Telemetry Transport. Lightweight publish-subscribe messaging protocol.
Mosquitto	Open-source MQTT broker. Runs on the Raspberry Pi.
Bridge	`bridge.py` — Python service on RPi that relays data between MQTT and Firebase.
Firebase	Google Cloud platform. MMS V2 uses Firestore, RTDB, Storage, Hosting, and Auth.
Firestore	Firebase document database. Stores permanent records (users, sessions, machines).
RTDB	Firebase Realtime Database. Stores live state (node status, pending commands).
LittleFS	Lightweight filesystem for flash memory. Used on ESP32 for offline session buffering.
NVS	Non-Volatile Storage. ESP32's key-value store. Used to persist configuration.
OTA	Over-The-Air firmware update.
HMAC-SHA256	Hash-based Message Authentication Code using SHA-256. Used for key derivation and PIN hashing.
Sector key	6-byte key that unlocks a MIFARE sector for read/write. Derived per-card using HMAC-SHA256.
Permission bitmask	32-bit unsigned integer. Each bit represents access to one machine.

Term	Definition
HC89	Optical IR slot sensor. Detects when a card is inserted.
Relay	Electrically controlled switch. Opens/closes machine power circuit.
SSR	Solid-State Relay. Semiconductor-based relay, no moving parts. Preferred for high-cycling loads.
WS2812	Individually addressable RGB LED with integrated controller chip.
SPI	Serial Peripheral Interface. Communication bus shared by RFID reader and primary OLED.
I2C	Inter-Integrated Circuit. Two-wire bus used by secondary OLED.
IST	Indian Standard Time. UTC+5:30. Used for all NTP-synced timestamps on nodes.
TWDT	Task Watchdog Timer. 30-second hardware watchdog on ESP32. Resets if firmware stalls.

Chapter 2

History and V1 Post-Mortem

2.1 V1 System Background

The original Machine Management System (V1) was built as a student project to introduce RFID-based access control to the Tinkerers' Lab. The V1 system used ESP32 microcontrollers with MFRC522 RFID readers and a Flask-based kiosk to issue cards. However, the system couldn't worked end-to-end and was never deployed in production.

A detailed audit of V1 identified multiple critical failures across the firmware, kiosk, web application, and backend layers.

2.2 V1 Failure Analysis

Table 2.1: V1 Critical Bugs and Root Causes

#	Bug	Effect	Root Cause
1	RFID block mismatch	Kiosk writes blocks 1+2; nodes read block 5 — no valid read ever possible	Copy-paste error; never tested end-to-end
2	Unhashed PINs	PINs stored in plaintext on card and Firestore	No security review
3	Non-unique MQTT client IDs	All nodes use same ID; broker disconnects the previous when new node connects; only one node online at a time	MQTT spec not understood
4	No watchdog	Firmware hangs cause node to stay in relay-ON state indefinitely	Not implemented
5	Blocking `delay()` calls	Display updates and RFID polling compete; 200ms RFID scan blocks all other operations	Arduino delay() used in production code
6	Wrong MFRC522 RST pin	RST on GPIO 34 (input-only on ESP32); can never be driven LOW; RFID module never resets correctly	ESP32 GPIO capabilities not checked
7	Firestore: `addDoc` for users	User documents get auto-generated IDs instead of roll number as document ID; lookup by roll number impossible	Firebase API misuse
8	No MQTT→Firebase bridge	Session events published to MQTT never reach Firestore	Missing architectural component
9	Machine name mismatch	`config.h` said "Soldering Station 1" for what is actually the 3D Printer	Manual entry error

2.3 V1 Deployment Blockers

Beyond the bugs above, V1 had architectural gaps that would prevent reliable deployment even if the bugs were fixed:

- No offline buffering: if WiFi is down during a session, the event is lost
- No OTA: firmware updates require physical access to each node

- No card revocation: a lost card cannot be invalidated without re-issuing every card
- No session timeout: a student who leaves without removing their card causes the machine to run indefinitely
- No admin remote control: no way to stop a machine remotely
- No stale command handling: a stop command stored when a node is offline would be executed on reconnect regardless of whether a session is active

2.4 V2 Design Decisions Driven by V1 Failures

Each major V2 design decision directly addresses a V1 failure:

New RFID card schema (Sector 1, Blocks 4+5) Fixes bug #1. The schema is explicitly versioned (schema byte 0x02) so firmware can detect and reject V1 cards.

HMAC-SHA256 PIN hashing Fixes bug #2. PIN is never stored in plaintext anywhere. The hash is truncated to 8 bytes (64 bits) — sufficient for a 4-digit PIN with rate limiting.

Per-node unique MQTT client ID: `mms_node_{id}` Fixes bug #3. Client IDs are derived from the machine ID, guaranteed unique across all nodes.

30-second TWDT hardware watchdog Fixes bug #4. If firmware stalls for any reason, the ESP32 resets within 30 seconds.

Non-blocking millis()-based state machine Fixes bug #5. No `delay()` calls in production firmware. All timing is event-driven with `millis()` comparisons.

MFRC522 RST moved to GPIO 27 Fixes bug #6. GPIO 27 is a standard bidirectional GPIO that can be driven both HIGH and LOW.

Firestore setDoc with roll number as key Fixes bug #7. `/users/{roll_number}` is the document path; lookup by roll number is O(1).

bridge.py MQTT→Firebase bridge Fixes bug #8. All session events flow: ESP32 → MQTT → bridge.py → Firestore. Bridge runs as a systemd service with automatic restart.

Canonical machine registry in three files Fixes bug #9. Machine names are documented in a central table. Any mismatch between config.h, seed_machines.py, and config.py is a bug.

2.5 Decision Records

The following key architectural decisions were made during V2 design and are recorded here for future maintainers.

2.5.1 Local Access Decision (No Cloud Call on Access Path)

Access decisions (grant/deny) are made entirely on the ESP32, using card data read from the RFID chip and a revocation list cached in LittleFS. This means:

- Access works if WiFi is down
- Access works if Firebase is down
- Access works if the RPi is down
- Latency of the access decision is deterministic (~100ms)

The trade-off is that card revocation can be delayed: a newly revoked card is denied only after the revocation list is pushed via MQTT and received by the node. If the node is offline, the revoked card may be accepted until the node reconnects.

2.5.2 Card Carries Permissions

Machine permissions are stored on the physical RFID card (Block 5, bytes 8–11, uint32 bitmask). The node does not look up permissions in Firebase or any network service. This means:

- Permission changes require a card re-issue
- Compromised cards carry wrong permissions until revoked
- No network call needed for the access decision

Alternative considered: store permissions in Firebase and look them up on each tap. Rejected because it breaks offline operation.

2.5.3 Per-Card Sector Key Derivation

Each RFID card's Sector 1 is protected by a unique 6-byte key derived as:

$$\text{sector\_key}[6] = \text{HMAC-SHA256}(\text{master\_key}, \text{uid\_hex\_uppercase})[0..5]$$

This ensures that knowledge of one card's sector key does not help an attacker forge another card. The master key is the only secret that must be protected system-wide.

2.5.4 5-Minute Command TTL

Remote stop commands have a 5-minute TTL. A command older than 300 seconds is discarded without execution. This prevents a node from executing a stop command that was issued while it was offline, after it reconnects and the context has changed.

2.5.5 MQTT Buffer Size 1536 Bytes

PubSubClient's default buffer is 256 bytes, which is insufficient for MQTT messages containing the full revocation list (multiple UIDs as JSON). The buffer was increased to 1536 bytes in `mqtt_client.cpp`.

Chapter 3

System Architecture

3.1 Architecture Overview

MMS V2 uses a three-tier architecture:

1. **Edge tier**: ESP32 access nodes. One per machine. Physical access control, session timing, display.
2. **Local bridge tier**: Raspberry Pi running Mosquitto (MQTT broker) and `bridge.py`. Bridges the local MQTT network and Firebase.
3. **Cloud tier**: Firebase (Firestore + RTDB + Storage + Hosting + Auth). Permanent storage, real-time state, web hosting.

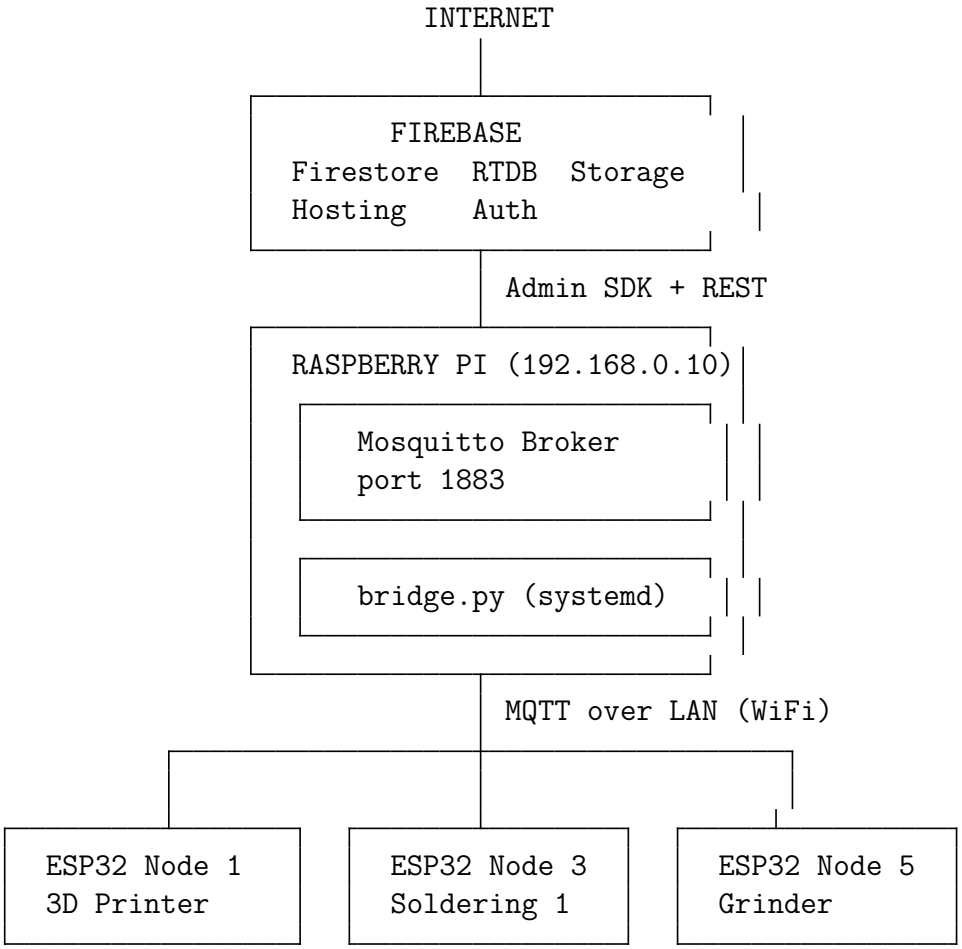


Figure 3.1: MMS V2 System Architecture

3.2 Communication Paths

3.2.1 Access Decision (Offline-capable)

The access decision path requires no network:

1. HC89 slot sensor detects card insertion
2. MFRC522 reads card UID
3. ESP32 derives sector key: HMAC-SHA256(master_key, uid_hex)[0..5]
4. ESP32 authenticates Sector 1, reads Blocks 4 and 5
5. ESP32 checks UID against LittleFS revocation list
6. ESP32 checks permission bitmask: $(perm\_mask \gg (machine\_id - 1))\&1$
7. If all checks pass: relay ON, session starts

3.2.2 Session Event Flow

1. Session ends (card removed, timeout, or remote stop)
2. ESP32 builds JSON event: `{ts, roll, machine_id, event, duration_s, end_reason}`
3. If MQTT connected: publish to `mms/nodes/{id}/event` (QoS 1)
4. If MQTT disconnected: append to LittleFS `/logs/sessions.jsonl`; publish on next connect
5. bridge.py receives event via MQTT subscription
6. bridge.py checks Firestore for duplicate (±5s deduplication window)
7. bridge.py writes session to Firestore `/sessions` collection
8. bridge.py updates `/users/{roll}/last_seen`

3.2.3 Remote Stop Flow

1. Admin clicks Stop in web dashboard
2. Web app writes to RTDB `/mms/commands/{id}`: `{command:"stop", timestamp:now}`
3. bridge.py `onValue` listener triggers
4. bridge.py checks command age: if >300s, discard and acknowledge
5. bridge.py publishes to `mms/nodes/{id}/command` (QoS 1)
6. ESP32 MQTT callback receives command
7. ESP32 checks command timestamp: if >300s old, discard
8. ESP32 turns relay OFF, logs end_reason: `remote_stop`
9. ESP32 publishes session event; acknowledges command to RTDB

3.2.4 Revocation Flow

1. Admin marks card as lost in web app
2. Web app creates `/revoked/{uid}` in Firestore
3. bridge.py Firestore `onSnapshot` triggers
4. bridge.py rebuilds revocation list from all `/revoked` documents
5. bridge.py publishes `mms/all/revoked` (retained, QoS 1): `{uids:[...], updated_at:ts}`
6. All connected nodes receive the update; update LittleFS `/config/revoked.json`
7. On next card tap: UID checked against revocation list → denied
8. Offline nodes: receive update on reconnect (retained message delivered automatically)

3.2.5 OTA Update Flow

1. Super admin uploads `.bin` to Firebase Storage; copies download URL
2. Super admin writes to RTDB `/mms/ota`: `{firmware_url, target_version, released_at}`
3. bridge.py RTDB OTA listener triggers
4. bridge.py publishes OTA command to all nodes: `mms/nodes/+/command`
5. ESP32 receives command; transitions to OTA state; relay OFF
6. ESP32 calls `esp_https_ota()` with firmware URL
7. On success: reboot to new partition; update NVS fw_version
8. On failure: if 3 crashes on new partition → rollback to previous partition

3.3 MQTT Topic Map

Topic	Publisher	QoS	Retain	Payload
`mms/nodes/{id}/status`	ESP32	1	Yes	State, roll, session_start, fw_version, IP, RSSI
`mms/nodes/{id}/event`	ESP32	1	No	Session event JSON
`mms/nodes/{id}/heartbeat`	ESP32	0	No	{ts, heap_free, uptime_s}
`mms/nodes/{id}/command`	RPi bridge	1	No	{cmd, issued_by, ts}
`mms/nodes/{id}/config`	RPi bridge	1	Yes	{name, session_limit, relay_active_high, machine_active}
`mms/all/revoked`	RPi bridge	1	Yes	{uids:[...], updated_at}

Table 3.1: MQTT Topic Reference

Last Will and Testament (LWT): Each ESP32 registers an LWT on `mms/nodes/{id}/status` with payload `{"state":"offline"}` (retained). If the node disconnects unexpectedly, the broker publishes this automatically. The bridge detects it and updates the RTDB node state to `"offline"`, which the dashboard reflects immediately.

3.4 Offline Degradation Table

Table 3.2: System Behaviour When Components Are Unavailable

Unavailable component	**Impact on access control**	**Impact on logging**
WiFi (node disconnected)	No impact. Card access continues.	Sessions buffered in LittleFS; flushed on reconnect
Raspberry Pi down	No impact. Access decisions are local.	Sessions buffered; flushed when RPi recovers
Firebase down	No impact.	bridge.py buffers events in memory; written when Firebase recovers
RFID reader (hardware fault)	No card reads possible. Node logs error.	No sessions started.
Relay (hardware fault)	Machine may not respond to access grant.	Sessions still logged.

Chapter 4

Hardware Design

4.1 ESP32 NodeMCU 38-pin

The ESP32 NodeMCU (38-pin variant) is the central controller for each access node. Key specifications relevant to MMS V2:

- Dual-core Xtensa LX6 at up to 240 MHz
- 520 KB SRAM; 4 MB flash
- Built-in 802.11 b/g/n WiFi (2.4 GHz only)
- 34 programmable GPIOs; some input-only
- Hardware SPI, I2C, UART peripherals
- `mbedTLS` cryptography library in ROM (provides HMAC-SHA256 without adding flash)
- Dual OTA flash partitions; watchdog timer
- 3.3V operating voltage; 5V tolerant on Vin pin

GPIO 34–39 are INPUT ONLY. They cannot be driven as outputs and have no internal pull-up resistors. In V1, the MFRC522 RST pin was connected to GPIO 34, which cannot be driven LOW — the reset never worked. V2 moves RST to GPIO 27. The rotary encoder CLK (GPIO 34) and DT (GPIO 35) are correctly used as inputs with external 10kΩ pull-up resistors to 3.3V.

4.2 GPIO Pin Assignments

Table 4.1: ESP32 GPIO Assignments for MMS V2 Access Node

GPIO	Signal	Direction	Pull resistor	Notes
5	MFRC522 SS (SDA)	OUT	None	SPI chip select
18	SPI SCK	OUT	None	Shared: RFID + OLED64
19	SPI MISO	IN	None	Shared: RFID + OLED64
23	SPI MOSI	OUT	None	Shared: RFID + OLED64
27	MFRC522 RST	OUT	None	Active LOW reset
17	OLED64 DC	OUT	None	Data/Command select
16	OLED64 CS	OUT	None	SPI chip select
4	OLED64 RST	OUT	None	Active LOW reset
21	OLED32 SDA	BIDIR	Module 4.7kΩ	I2C data
22	OLED32 SCL	OUT	Module 4.7kΩ	I2C clock
26	Relay IN	OUT	None	LOW before `pinMode`
32	HC89 slot sensor	IN	INPUT_PULLUP	LOW = card present
33	WS2812 DIN	OUT	None	Via 470Ω resistor
34	Encoder CLK	IN	EXT 10kΩ to 3V3	Input-only; no internal pull-up
35	Encoder DT	IN	EXT 10kΩ to 3V3	Input-only; no internal pull-up
25	Encoder SW	IN	INPUT_PULLUP	Button press
2	Onboard LED	OUT	None	WiFi status indicator

4.3 Component Details

4.3.1 MFRC522 RFID Reader

The MFRC522 is a 13.56 MHz RFID reader module compatible with MIFARE cards. In MMS V2 it is connected via SPI.

- **Supply voltage**: 3.3V (NOT 5V — module is not 5V tolerant)
- **SPI speed**: up to 10 MHz; MMS firmware uses 4 MHz for stability
- **Pin labelling**: the "SDA" pin on the module is the SPI chip select (SS), not I2C SDA — this is a common source of confusion

- **IRQ pin**: not used in MMS V2; leave unconnected (NC)
- **Card detection**: firmware polls `MFRC522::PICC_IsNewCardPresent()` rather than using the IRQ line

4.3.2 SSD1306 128×64 SPI OLED (Primary Display)

This is the main user-facing display. It shows machine state, roll number, session timer, and status messages.

- Controller: SSD1306
- Resolution: 128×64 pixels, monochrome
- Interface: SPI (4-wire: SCK, MOSI, DC, CS) plus RST
- Supply: 3.3V
- Shared SPI bus with MFRC522; multiplexed via chip select pins

4.3.3 SSD1306 128×32 I2C OLED (Status Strip)

This is the always-visible status display: WiFi/MQTT connectivity dots, machine name, NTP time.

- Controller: SSD1306
- Resolution: 128×32 pixels, monochrome
- Interface: I2C, address 0x3C
- Supply: 3.3V
- Module has onboard 4.7kΩ pull-ups on SDA and SCL — no external resistors needed

4.3.4 HC89 Optical IR Slot Sensor

The HC89 is an infrared photointerrupter slot sensor that detects card insertion.

- Slot width: ~6mm (MIFARE card thickness: 0.76mm — fits well)
- Output type: open-collector (active LOW when card blocks IR beam)
- Supply: 3.3V or 5V (check module datasheet — most accept 3.3V)
- Configured as `INPUT_PULLUP` on GPIO 32
- Firmware debounce: 50ms (ignores transitions shorter than 50ms)
- A card in the slot gives signal LOW; no card gives signal HIGH

4.3.5 Relay Module (Mechanical or SSR)

Mechanical Relay

- Typical: SRD-05VDC-SL-C (5V coil, 10A 250V AC rated)
- Control: GPIO 26 via optocoupler on relay board
- Default active HIGH (`relay_active_high = true` in config)
- Relay board takes 5V on VCC; control signal from 3.3V GPIO is sufficient via optocoupler

Solid-State Relay (SSR)

Recommended for high-cycling loads (soldering station on/off frequently) and for the laser cutter (noise sensitivity).

- Typical: SSR-25DA (3–32V DC control, 25A 380V AC load)
- Control: GPIO 26 (3.3V signal drives 3–32V DC input directly)
- Active HIGH (`relay_active_high = true`)
- Heat sink required for loads above 10A

> The `relay_active_high` configuration is pushed from Firebase via MQTT to each node's NVS. This means the relay polarity can be changed remotely without reflashing. Set this value correctly in the Firestore `/machines/{id}` document when seeding.

4.3.6 WS2812B RGB LED

A single WS2812B LED provides colour-coded system state feedback.

- Supply: 5V (directly from Vin — NOT 3.3V)
- Data: GPIO 33 via 470Ω series resistor (protects against inductive spikes from long wire runs)
- 100μF electrolytic capacitor across VCC–GND at the LED (absorbs power-up surge)
- Colour order: `NEO_GRB` (some clone modules use different order — see Appendix G)

4.3.7 EC11 Rotary Encoder

The EC11 rotary encoder is installed in the hardware but not yet used in production firmware (Phase 4 — PIN entry at machine).

- CLK on GPIO 34 (input-only) — requires external 10kΩ pull-up to 3.3V
- DT on GPIO 35 (input-only) — requires external 10kΩ pull-up to 3.3V
- SW (push button) on GPIO 25 — uses internal `INPUT_PULLUP`

4.3.8 Power Supply

Each node uses a HiLink HLK-PM01 (5V 1W AC-DC converter) to derive 5V from mains:

- Input: 90–264V AC
- Output: 5V DC, 600mA
- The 5V feeds: ESP32 Vin, relay module VCC, WS2812 VCC
- The ESP32's internal LDO produces 3.3V for all other components
- A 5A fuse must be placed on the AC input before the HiLink

Safety: The HiLink operates at 240V AC. Treat it as a live shock hazard at all times. All AC wiring must be completed, routed through cable glands, and the enclosure must be closed before the node is powered from mains. Always test 5V DC circuitry with a USB power bank before connecting to mains.

4.4 Power Budget

Table 4.2: Node Power Consumption Estimate

Component	Typical (mA)	Peak (mA)
ESP32 (WiFi active, 240MHz)	160	250
MFRC522	13	26
SSD1306 64px OLED	20	25
SSD1306 32px OLED	15	20
HC89	20	20
WS2812B (1 LED, white full)	60	60
Relay coil (mechanical)	70	70
Total (peak)		**471 mA**
HiLink HLK-PM01 capacity		600 mA
Margin		127 mA (21%)

The power margin is adequate but not generous. Avoid adding components to the 5V rail beyond the designed BOM. If an SSR is added (replacing mechanical relay), remove the relay coil consumption — SSRs consume ~10mA on the control side.

Chapter 5

RFID Card Architecture

5.1 MIFARE Classic 1K Overview

MIFARE Classic 1K cards have 1024 bytes of non-volatile EEPROM organised as:

- 16 sectors (sectors 0–15)
- 4 blocks per sector (each block = 16 bytes)
- Block 0 of Sector 0 is the manufacturer block (UID + config; read-only)
- Each sector has a trailer block (last block in sector) containing two keys (Key A and Key B) and access bits
- Data blocks: 3 per sector (excluding trailer)
- Sectors are authenticated independently using their own keys

5.2 V2 Card Schema

MMS V2 uses **Sector 1** (blocks 4–7). Sector 0 is left untouched (manufacturer data and default keys). Using Sector 1 means V2 cards can coexist with other applications on Sector 0 if needed.

5.2.1 Sector 1 Layout

Table 5.1: MMS V2 RFID Card Schema — Sector 1

Block	Bytes	Content
4	0–8	Roll number as ASCII string, zero-padded to 9 characters
	9	Schema version byte: `0x02` (V2)
	10–15	Reserved, all `0x00`
5	0–7	PIN hash: HMAC-SHA256(master_key‖uid_hex, pin_string)[0..7]
	8–11	Machine permission bitmask (uint32, little-endian)
	12–15	Reserved, all `0x00`
6	0–15	Reserved for future use (expiry date, academic year)
7	0–5	Key A = derived sector key (write-only; never returned by MFRC522 on read)
	6–9	Access bits
	10–15	Key B = `0x00` (not used)

5.2.2 Schema Version Byte

The schema version byte at Block 4, byte 9 is `0x02` for all V2 cards. If a card is read and this byte is not `0x02`, the firmware rejects it with `ACCESS_DENIED` and logs `"schema version mismatch"`. This prevents:

- V1 cards (written to Block 1+2 with no schema byte) from being accepted
- Blank or foreign cards from accidentally gaining access
- Future V3 cards (with schema byte `0x03`) from being accepted by V2 nodes

5.3 Sector Key Derivation

Each card's Sector 1 is protected by a unique 6-byte key derived from the master key and the card's UID:

$$\text{sector\_key}[6] = \text{HMAC-SHA256}(\text{master\_key}, \text{uid\_as\_uppercase\_hex\_string})\,[0..5]$$

For a card with UID bytes `[0xA1, 0xB2, 0xC3, 0xD4]`, the UID string is `"A1B2C3D4"`. This derivation is performed by:

- The kiosk (`station_writer.ino`) when writing the card, to set the sector key
- Every access node on every card read, to authenticate Sector 1 before reading blocks

The master key is identical across all nodes and the kiosk — they all share the same 32-byte secret stored in `secrets.h` / `secrets.py` / `secrets_station.h`.

HMAC-SHA256 is implemented using `mbedtls_md_hmac()` from the ESP32's built-in mbedTLS library. No additional library is required. The same function is called from the kiosk (C++ Arduino) and the RPi bridge (Python `hmac` module with sha256 digest).

5.4 PIN Hash

The PIN hash stored in Block 5 bytes 0–7 is:

$$\text{pin\_hash}[8] = \text{HMAC-SHA256}(\text{master\_key} \| \text{uid\_hex},\ \text{pin\_string})\,[0..7]$$

Where `pin_string` is the 4-character ASCII PIN (e.g., "1234").
The PIN hash is computed by:

- The kiosk when issuing the card
- The web app's `hashPin()` function when the admin sets a PIN
- Access nodes do NOT verify PIN in V2 (Phase 4)

In V2, the PIN is not verified at the machine. The PIN hash is stored on the card for future use (Phase 4 — rotary encoder PIN entry). In V2, card tap + permission bit check is sufficient for access.

5.5 Machine Permission Bitmask

Block 5 bytes 8–11 contain a 32-bit little-endian unsigned integer. Each bit corresponds to one machine:

$$\text{allowed} = (\text{perm\_bitmask} \gg (\text{machine\_id} - 1)) \ \&\ 1 \tag{5.1}$$

Table 5.2: Machine Permission Bitmask Examples

Access to machines	Decimal	Hex	Binary (LSB first)
None (no access)	0	`0x00000000`	`00000000...`
Machine 1 only	1	`0x00000001`	`10000000...` (bit 0 set)
Machine 3 only	4	`0x00000004`	`00100000...` (bit 2 set)
Machines 1 and 3	5	`0x00000005`	`10100000...` (bits 0+2 set)
All 5 machines	31	`0x0000001F`	`11111000...` (bits 0–4 set)
Full access (all 32)	4294967295	`0xFFFFFFFF`	all bits set

The bitmask supports up to 32 machines. Currently 5 are deployed (bits 0–4). Bits 5–31 are reserved for future machines.

5.6 Card Write Process

When the kiosk writes a card:

1. Admin selects student and machine permissions in web UI
2. Flask app computes bitmask and PIN hash
3. Flask app sends JSON to station_writer via serial:
   `{"roll":"220002123","perm_mask":5,"pin_hash":"aabb...","uid":"A1B2C3D4"}`
4. station_writer.ino receives JSON (ArduinoJson), places card on MFRC522
5. Derives sector key from UID
6. Authenticates Sector 1 with default key (`0xFFFFFFFFFFFF` for fresh cards) or derived key (for re-issue)
7. Writes Block 4: roll bytes 0–8 + schema byte `0x02` + zeros 10–15
8. Writes Block 5: pin_hash bytes 0–7 + perm_mask LE bytes 8–11 + zeros 12–15
9. Writes Sector 1 trailer (Block 7): Key A = derived sector key, Key B = zeros
10. Reads back Block 4 and verifies schema byte = `0x02`
11. Reports `WRITE_SUCCESS` or `WRITE_FAILED` via serial to Flask app

5.7 Card Read Process

When a node reads a card during an access attempt:

1. HC89 signals card present
2. `MFRC522::PICC_IsNewCardPresent()` confirms a card in field
3. `MFRC522::PICC_ReadCardSerial()` reads the 4-byte UID
4. Sector key is derived from UID + master key
5. `MFRC522::PCD_Authenticate()` authenticates Sector 1 with derived key
6. Block 4 is read; schema byte checked (must be `0x02`)
7. Block 5 is read; perm_mask extracted from bytes 8–11 (LE decode)
8. UID is checked against LittleFS revocation list
9. Permission bit checked for this machine_id
10. Result: GRANTED or DENIED with reason

Chapter 6

Firmware Architecture

6.1 Firmware Structure

The access node firmware is located in `v2/access_node/`. It is organised as a collection of single-responsibility modules coordinated by a central state machine.

Table 6.1: Firmware Module Map

Module	Files	Responsibility
Config	`config.h`	Machine ID, name, session limit, broker IP
Secrets	`secrets.h`	WiFi credentials, master key (gitignored)
NVS Manager	`nvs_manager.h/.cpp`	Read/write NVS namespace `mms_config`
RFID Handler	`rfid_handler.h/.cpp`	MFRC522 SPI init; card detect; sector key derivation; block read
Card Schema	`card_schema.h/.cpp`	Parse blocks 4+5; validate schema; return `CardData` struct
Access Control	`access_control.h/.cpp`	Revocation list check + permission bitmask check
Relay Controller	`relay_controller.h/.cpp`	GPIO abstraction; safe OFF on init
Slot Sensor	`slot_sensor.h/.cpp`	HC89 debounced input
Session Manager	`session_manager.h/.cpp`	Session start/end/timeout; build log entry JSON
MQTT Client	`mqtt_client.h/.cpp`	PubSubClient wrapper; reconnect backoff; command callback
Revocation Cache	`revocation_cache.h/.cpp`	LittleFS `/config/revoked.json`; UID lookup
LittleFS Logger	`littlefs_logger.h/.cpp`	JSONL append; rotate at 50KB; flush on connect
Display Primary	`display_primary.h/.cpp`	128×64 SPI OLED; state-driven rendering
Display Status	`display_status.h/.cpp`	128×32 I2C OLED; persistent status strip
LED Controller	`led_controller.h/.cpp`	WS2812; non-blocking animations
Watchdog	`watchdog.h/.cpp`	ESP32 TWDT; 30s timeout; feed in main loop
OTA Handler	`ota_handler.h/.cpp`	Version check; `esp_https_ota`; rollback
State Machine	`state_machine.h/.cpp`	Central FSM; drives all transitions
Main	`access_node.ino`	`setup()` + `loop()`; watchdog feed

6.2 Firmware State Machine

The state machine is the central coordinator. All hardware events and MQTT messages cause state transitions. No module changes hardware state directly — all changes go through the state machine.

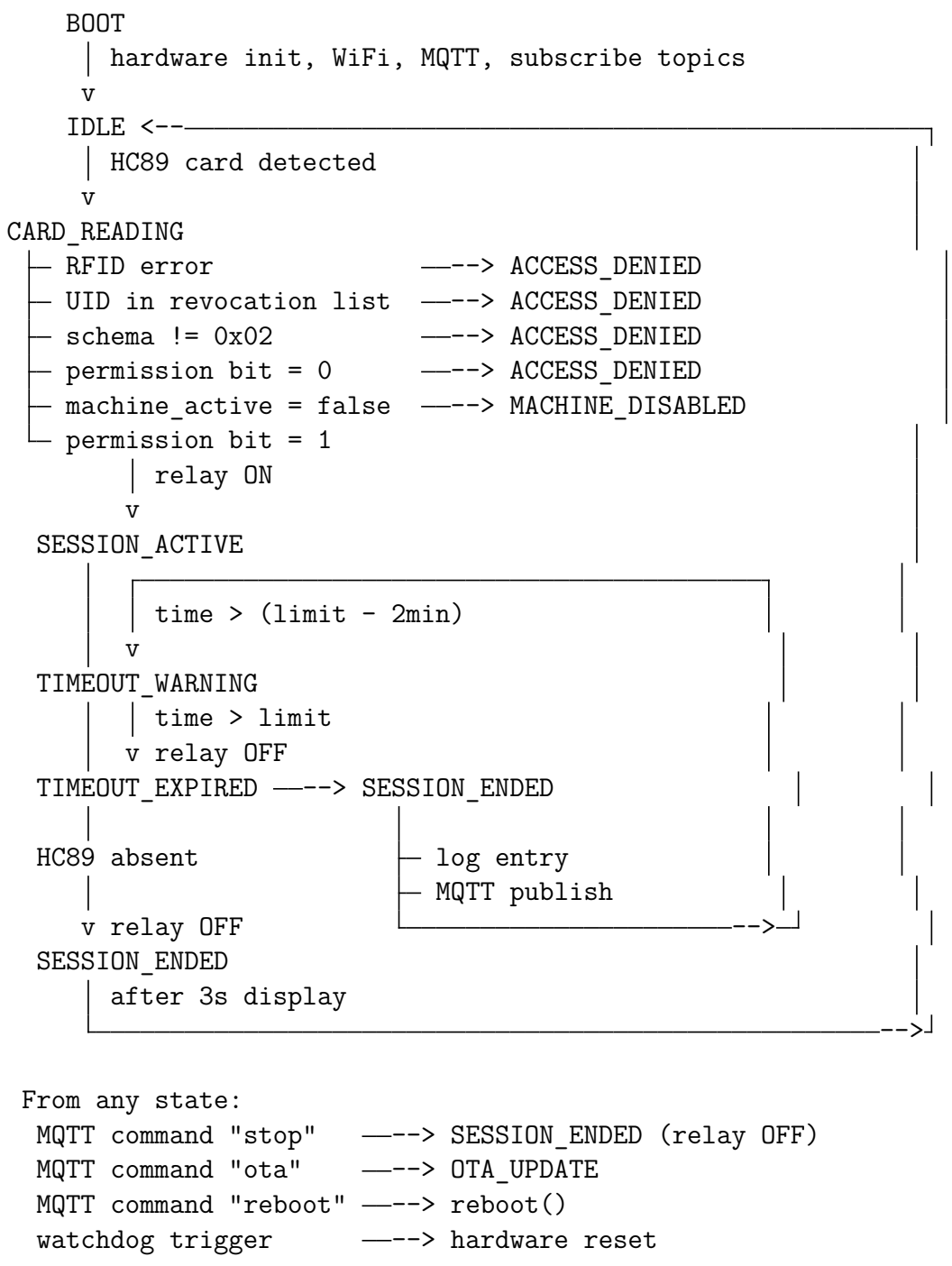


Figure 6.1: Access Node Firmware State Machine

6.3 NVS Configuration

NVS (Non-Volatile Storage) persists configuration across reboots. The namespace is `mms_config`.

Table 6.2: NVS Configuration Keys

Key	Type	Set by	Description
`machine_id`	uint8	`config.h` at flash	1–32
`master_key`	blob (32B)	`secrets.h` at flash	HMAC master secret
`machine_name`	string (32)	MQTT config push	Display name; fallback to `config.h`
`session_limit`	uint16	MQTT config push	Minutes; fallback to `config.h`
`relay_active_high`	uint8	MQTT config push	1=HIGH activates relay; 0=LOW
`machine_active`	uint8	MQTT config push	0=disabled; all access denied
`fw_version`	string	OTA handler	Current firmware version string

On first boot (no NVS data), the firmware uses the fallback values from `config.h`. On the first MQTT connect, the bridge pushes the Firestore config, which overwrites NVS. On subsequent boots without MQTT connection, NVS values are used.

6.4 LittleFS File Layout

```
/logs/
  sessions.jsonl      <- append-only event log
  sessions.old.jsonl  <- previous rotation backup

/config/
  revoked.json        <- {uids:["A1B2C3",...], updated_at:unix_ts}
```

Log rotation: When `sessions.jsonl` exceeds 50KB:

1. Rename `sessions.jsonl` → `sessions.old.jsonl` (overwrites previous)
2. Create new empty `sessions.jsonl`
3. Continue appending

Flush on MQTT connect:

1. Open `sessions.jsonl`
2. Read line by line; attempt JSON parse each line

3. Discard any line that fails to parse (partial write from power loss)
4. Publish each valid line to `mms/nodes/{id}/event` (QoS 1)
5. Wait for broker ACK before publishing next
6. After all lines published: delete file

6.5 Session Log Entry Format

Each line in `sessions.jsonl` is a compact JSON object:

```
{"t":1718600000,"r":"220002123","m":3,"e":"session_end",
 "d":245,"er":"card_removed"}
```

Listing 6.1: Session log entry format

Fields:

- `t`: Unix timestamp of session end
- `r`: Roll number string
- `m`: Machine ID integer
- `e`: Event type (`"session_end"`, `"boot"`)
- `d`: Duration in seconds
- `er`: End reason (`"card_removed"`, `"timeout"`, `"remote_stop"`, `"watchdog"`)

6.6 Display Content

6.6.1 Primary OLED (128×64)

Table 6.3: Primary Display Content by State

State	Display content
IDLE	"Tinkerers' Lab / IIT Indore" + machine name + "Tap card to start"
CARD_READING	"Reading card..." + spinner animation
ACCESS_DENIED	"!! Access Denied !!" + reason + roll number (clears after 3s)
MACHINE_DISABLED	"Machine Offline" + "Contact admin"
SESSION_ACTIVE	Roll number + machine name + "Time: MM:SS / HH:MM"
TIMEOUT_WARNING	"!! 2 min left !!" + "Remove card soon" + roll number
SESSION_ENDED	"Session Complete" + roll + duration + "Thank you!"
OTA_UPDATE	"Updating FW..." + version + progress percentage + "Do not remove"
ERROR	"System Error" + error code + "Restarting..."

6.6.2 Status Strip OLED (128×32)

Always shows:

```
[WiFi*] [MQTT*]   MACHINE NAME   14:58
```

WiFi and MQTT indicators are filled circles when connected; empty circles when not.

6.7 WS2812 LED State Map

Table 6.4: LED Colour and Pattern per State

State	Colour	Pattern
IDLE	White (dim)	Slow breathe, 2s period
CARD_READING	Cyan	Fast blink, 200ms
SESSION_ACTIVE	Green	Solid
TIMEOUT_WARNING	Yellow ↔ Orange	Alternate, 1s
TIMEOUT_EXPIRED	Red	Solid (500ms)
ACCESS_DENIED	Red	3× quick flash then off
MACHINE_DISABLED	Purple	Solid dim
OTA_UPDATE	Blue	Chasing animation
ERROR	Red	Solid

6.8 Watchdog

The hardware watchdog is the last line of defence against firmware hangs:

```
esp_task_wdt_init(30, true);  // 30s timeout, panic on trigger
esp_task_wdt_add(NULL);       // watch the main task
```

Listing 6.2: Watchdog initialisation in setup()

```
void loop() {
    esp_task_wdt_reset();   // must be called at least every 30 s
    stateMachine.tick();
    // ...
}
```

Listing 6.3: Watchdog feed in main loop

If `loop()` takes longer than 30 seconds (e.g., blocking I2C, infinite RFID retry), the TWDT triggers a panic and the ESP32 reboots. The boot reason is available via `esp_reset_reason()` on the next boot.

Chapter 7

Backend Architecture

7.1 Raspberry Pi Role

The Raspberry Pi is the communication hub between the local MQTT network and Firebase cloud. It runs two services:

- **Mosquitto**: MQTT message broker; all ESP32 nodes connect to it
- **bridge.py**: Python service that subscribes to MQTT topics and synchronises data with Firebase

The RPi is connected via Ethernet to the lab router and must have a static IP address (192.168.0.10 by default).

7.2 Mosquitto Configuration

Mosquitto is configured in `/etc/mosquitto/conf.d/mms.conf`:

```
listener 1883 0.0.0.0
allow_anonymous true          # change to false after adding
   auth
persistence true
persistence_location /var/lib/mosquitto/
log_dest syslog
log_type all
log_timestamp true
max_connections 50
max_queued_messages 1000
```

Listing 7.1: Mosquitto configuration file

Key settings:

- **Persistence**: Retained messages (config push, revocation list, node status) survive broker restarts. Nodes reconnecting after a broker restart immediately receive their current config and revocation list.
- **max_connections**: 50 is sufficient for 15 nodes + 1 bridge + 5 admin tools.
- **allow_anonymous**: Set to `true` during initial deployment. Should be changed to `false` with password authentication after all nodes are confirmed working. See Section 7.9.

7.3 bridge.py Architecture

`bridge.py` is a single-process Python script that runs the following listeners concurrently using Paho MQTT's event loop and Firebase SDK callbacks:

1. **MQTT subscriber:** `mms/nodes/+/event` → write session to Firestore
2. **MQTT subscriber:** `mms/nodes/+/status` → update RTDB node state
3. **RTDB listener:** `/mms/commands/{id}` → forward commands to MQTT
4. **RTDB listener:** `/mms/ota` → broadcast OTA command to all nodes
5. **Firestore listener:** `/machines/*` → push config to nodes via retained MQTT
6. **Firestore listener:** `/revoked` → push revocation list to `mms/all/revoked`
7. **Periodic sweep (every 10 min)** → clean up stale unacknowledged RTDB commands

7.4 Session Event Processing

When bridge.py receives an event on `mms/nodes/{id}/event`:

```
def on_event(client, userdata, message):
    event = json.loads(message.payload)
    machine_id = extract_machine_id(message.topic)

    # Deduplication: check for existing session within +/-5s
    start_ts = event["t"] - event["d"]
    existing = db.collection("sessions") \
        .where("machine_id", "==", str(machine_id)) \
        .where("start_time", ">=", datetime.fromtimestamp(
   start_ts - 5)) \
        .where("start_time", "<=", datetime.fromtimestamp(
   start_ts + 5)) \
        .limit(1).get()

    if existing:
        logger.info(f"Duplicate session detected -- skipping")
        return

    # Get user info from Firestore
    user_doc = db.collection("users").document(event["r"]).get
   ()
    user_name = user_doc.get("name") if user_doc.exists else "
   Unknown"

    # Write session record
    session = {
        "machine_id":   str(machine_id),
        "roll_number":  event["r"],
        "user_name":    user_name,
```

```
        "start_time":    datetime.fromtimestamp(event["t"] - event["d"]),
        "end_time":      datetime.fromtimestamp(event["t"]),
        "duration_s":    event["d"],
        "end_reason":    event["er"],
        "status":        "completed",
    }
    db.collection("sessions").add(session)

    # Update user last_seen
    db.collection("users").document(event["r"]).update(
        {"last_seen": firestore.SERVER_TIMESTAMP}
    )
```

Listing 7.2: Session event handler in bridge.py (simplified)

7.5 Command Forwarding

When bridge.py's RTDB listener fires for a command:

```
def on_commands_change(event):
    machine_id = extract_id_from_path(event.path)
    data = event.data

    if data is None or "command" not in data:
        return

    command = data["command"]
    ts = data.get("timestamp", 0)

    # Stale command check (5-minute TTL)
    if time.time() - ts > 300:
        logger.info(f"Stale command for machine {machine_id} -- discarding")
        rtdb.reference(f"mms/commands/{machine_id}").update({
            "acknowledged": True,
            "discarded": True,
            "reason": "stale_command",
        })
        return

    # Forward to node via MQTT
    payload = json.dumps({"cmd": command, "issued_by": data.get("issued_by"), "ts": ts})
    mqtt_client.publish(f"mms/nodes/{machine_id}/command", payload, qos=1)
```

Listing 7.3: Command forward handler in bridge.py (simplified)

7.6 Revocation List Management

The revocation list is maintained in Firestore `/revoked` collection. When any document is added or removed:

```python
def on_revoked_change(col_snapshot, changes, read_time):
    # Rebuild full list from snapshot
    uids = [doc.id for doc in col_snapshot]

    payload = json.dumps({
        "uids": uids,
        "updated_at": int(time.time())
    })

    # Publish retained -- nodes receive it on connect even if offline during update
    mqtt_client.publish("mms/all/revoked", payload, qos=1, retain=True)
    logger.info(f"Revocation list pushed: {len(uids)} UID(s)")
```

Listing 7.4: Revocation push in bridge.py (simplified)

7.7 Systemd Service

bridge.py runs as a systemd service with automatic restart:

```
[Unit]
Description=MMS Firebase Bridge
After=network-online.target mosquitto.service
Wants=network-online.target

[Service]
Type=notify
User=pi
WorkingDirectory=/home/pi/mms
EnvironmentFile=/home/pi/mms/.env
ExecStart=/home/pi/mms/venv/bin/python3 /home/pi/mms/bridge.py
Restart=always
RestartSec=5
StartLimitIntervalSec=60
StartLimitBurst=5
WatchdogSec=60
NotifyAccess=main

[Install]
WantedBy=multi-user.target
```

Listing 7.5: /etc/systemd/system/mms-bridge.service

Key settings:

- `Restart=always`: restart on any exit (crash, OOM, signal)

- `StartLimitBurst=5`: stop restarting after 5 failures in 60s (prevents runaway restart loop)
- `WatchdogSec=60`: systemd kills and restarts the bridge if it stops sending watchdog pings within 60s
- `EnvironmentFile`: loads `.env` before starting (MQTT credentials, Firebase URL)

7.8 Health Check Endpoint

bridge.py serves a minimal HTTP health check on port 8080:

```
curl http://192.168.0.10:8080/health
# Returns:
{"mqtt_connected": true, "fb_connected": true, "uptime_s":
   3600}
```

This endpoint can be polled by a monitoring system or checked manually to confirm the bridge is alive.

7.9 MQTT Authentication

In the initial deployment, `allow_anonymous true` is used to simplify setup. Once all nodes are confirmed working, authentication should be enabled:

```
# Create password file
sudo mosquitto_passwd -c /etc/mosquitto/passwd bridge
sudo mosquitto_passwd /etc/mosquitto/passwd node_1
# ... repeat for each node (node_1 through node_5)

# Update mms.conf:
# Change: allow_anonymous true
# To:     allow_anonymous false
#         password_file /etc/mosquitto/passwd

sudo systemctl restart mosquitto
```

Update each node's `secrets.h` with `MQTT_USER` and `MQTT_PASSWORD` and re-flash. For per-node topic ACLs (highly recommended before production), add an ACL file:

```
# Bridge: full access
user bridge
topic readwrite mms/#

# Per-node: scoped to own topics + global revocation
user node_1
topic write mms/nodes/1/status
topic write mms/nodes/1/event
topic write mms/nodes/1/heartbeat
topic read  mms/nodes/1/command
topic read  mms/nodes/1/config
topic read  mms/all/revoked
```

Listing 7.6: /etc/mosquitto/acl (per-node topic restrictions)

Without ACLs, a compromised node could issue commands to other nodes or trigger OTA for the entire lab.

Chapter 8

Firebase Design

8.1 Firebase Services Used

MMS V2 uses four Firebase services:

- **Firestore**: Permanent document database. Users, machines, sessions, revoked cards, admins.
- **Realtime Database (RTDB)**: Live state. Machine states, pending commands, OTA metadata, bridge heartbeat.
- **Storage**: Binary file hosting. Firmware `.bin` files for OTA.
- **Hosting**: Web app CDN hosting. Serves the React admin dashboard.
- **Authentication**: Admin user login. Email/password provider.

8.2 Why Two Databases?

Firestore and RTDB serve different purposes:

Table 8.1: Firestore vs RTDB usage in MMS V2

Aspect	Firestore	RTDB
Data model	Documents with subcollections	Flat JSON tree
Primary use	Historical records	Live state / commands
Write pattern	Write once, read rarely	Write frequently, overwrite
Pricing	Per read/write/delete	Per MB downloaded
Offline client support	Yes (Firebase SDK)	Yes (Firebase SDK)
Suitable for	Sessions, users, machines, revoked	Node status, commands, OTA

8.3 Firestore Schema

8.3.1 /users/{roll_number}

One document per student. Document ID is the 9-digit roll number.

```
{
  "roll_number":          "220002123",
  "name":                 "Student Name",
  "email":                "student@iiti.ac.in",
  "pin_hash":             "aabbccdd11223344",
  "card_uid":             "A1B2C3D4",
  "machine_permissions":  5,
  "issued_at":            "2024-06-01T10:00:00Z",
  "issued_by":            "admin_uid",
  "is_active":            true,
  "last_seen":            "2024-06-15T14:30:00Z"
}
```

Listing 8.1: /users/220002123

8.3.2 /machines/{machine_id}

One document per machine. Document ID is the string representation of the machine ID ("1", "2", etc.).

```
{
  "name":                  "Soldering Station 1",
  "session_limit_minutes": 15,
  "relay_active_high":     true,
  "machine_active":        true,
  "location":              "Zone C",
  "created_at":            "2024-06-01T00:00:00Z"
}
```

Listing 8.2: /machines/3

8.3.3 /sessions/{auto_id}

One document per completed session. Document ID is auto-generated by Firestore.

```
{
  "machine_id":   "3",
  "machine_name": "Soldering Station 1",
  "roll_number":  "220002123",
  "user_name":    "Student Name",
  "start_time":   "2024-06-15T10:00:00Z",
  "end_time":     "2024-06-15T10:15:00Z",
  "duration_s":   900,
  "end_reason":   "card_removed",
  "status":       "completed"
}
```

Listing 8.3: /sessions/auto-generated-id

Valid `end_reason` values: `card_removed`, `timeout`, `remote_stop`, `power_loss`, `watchdog`.

8.3.4 /revoked/{card_uid}

Document ID is the card UID hex string (e.g., "A1B2C3D4").

```
{
  "uid":          "A1B2C3D4",
  "roll_number": "220002123",
  "revoked_at":   "2024-06-15T12:00:00Z",
  "revoked_by":   "admin_uid",
  "reason":       "lost"
}
```

Listing 8.4: /revoked/A1B2C3D4

8.3.5 /admins/{uid} and /super_admins/{uid}

One document per admin user. Document ID is the Firebase Auth UID.

```
{
  "email":      "admin@iiti.ac.in",
  "name":       "Admin Name",
  "created_at": "2024-06-01T00:00:00Z",
  "is_active":  true
}
```

Listing 8.5: /admins/firebase-auth-uid

8.4 RTDB Schema

```
{
  "mms": {
    "nodes": {
      "1": {
        "state":            "active",
        "roll_number":      "220002123",
        "session_start":    1718600000,
        "firmware_version": "2.0.0",
        "ip_address":       "192.168.0.101",
        "last_seen":        1718600060,
        "rssi":             -55
      }
    },
    "commands": {
      "3": {
        "command":      "stop",
        "issued_by":    "admin_uid",
        "timestamp":    1718600100,
        "acknowledged": false
      }
    },
    "ota": {
```

```
      "firmware_url":   "https://firebasestorage.googleapis.com/...",
      "target_version": "2.1.0",
      "released_at":    1718600200
    },
    "revoked": {
      "uids":       ["A1B2C3D4", "E5F60718"],
      "updated_at": 1718600300
    },
    "bridge": {
      "status": {
        "state": "online",
        "ts":    1718600000
      }
    }
  }
}
```

Listing 8.6: Full RTDB structure

8.5 Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Sessions: read by admins, write NEVER from client (bridge uses Admin SDK)
    match /sessions/{sessionId} {
      allow read: if request.auth != null &&
        (exists(/databases/$(database)/documents/admins/$(request.auth.uid)) ||
         exists(/databases/$(database)/documents/super_admins/$(request.auth.uid)));
      allow write: if false;
    }

    // Users: admins can read and write
    match /users/{rollNumber} {
      allow read, write: if request.auth != null &&
        (exists(/databases/$(database)/documents/admins/$(request.auth.uid)) ||
         exists(/databases/$(database)/documents/super_admins/$(request.auth.uid)));
    }

    // Machines: admins can read and write
    match /machines/{machineId} {
      allow read, write: if request.auth != null &&
```

```
23          (exists(/databases/$(database)/documents/admins/$(
    request.auth.uid)) ||
24           exists(/databases/$(database)/documents/super_admins/$
    (request.auth.uid)));
25       }
26
27       // Revoked: admins can read and write
28       match /revoked/{uid} {
29         allow read, write: if request.auth != null &&
30           (exists(/databases/$(database)/documents/admins/$(
    request.auth.uid)) ||
31            exists(/databases/$(database)/documents/super_admins/$
    (request.auth.uid)));
32       }
33
34       // Admins: any authenticated user can read (for login check
    )
35       match /admins/{uid} {
36         allow read: if request.auth != null;
37         allow write: if request.auth != null &&
38           exists(/databases/$(database)/documents/super_admins/$(
    request.auth.uid));
39       }
40
41       // Super admins: super admin writes only
42       match /super_admins/{uid} {
43         allow read: if request.auth != null;
44         allow write: if request.auth != null &&
45           exists(/databases/$(database)/documents/super_admins/$(
    request.auth.uid));
46       }
47     }
48 }
```

Listing 8.7: v2/firestore.rules

8.6 Firestore Indexes

Three composite indexes are required for efficient queries on the sessions collection:

```
1  {
2    "indexes": [
3      {
4        "collectionGroup": "sessions",
5        "queryScope": "COLLECTION",
6        "fields": [
7          {"fieldPath": "machine_id", "order": "ASCENDING"},
8          {"fieldPath": "start_time", "order": "DESCENDING"}
9        ]
10     },
11     {
```

```
      "collectionGroup": "sessions",
      "queryScope": "COLLECTION",
      "fields": [
        {"fieldPath": "roll_number", "order": "ASCENDING"},
        {"fieldPath": "start_time", "order": "DESCENDING"}
      ]
    },
    {
      "collectionGroup": "sessions",
      "queryScope": "COLLECTION",
      "fields": [
        {"fieldPath": "status", "order": "ASCENDING"},
        {"fieldPath": "start_time", "order": "ASCENDING"}
      ]
    }
  ]
}
```

Listing 8.8: v2/firestore.indexes.json

Indexes must be deployed before the bridge can run session deduplication queries. Deploy with `firebase deploy -only firestore:indexes`. Building takes 5–10 minutes.

Chapter 9

Web Application

9.1 Overview

The web application is the administrator interface for MMS V2. It is a single-page application (SPA) built with React and TypeScript, using the Firebase SDK for Firestore and RTDB connectivity. It is deployed to Firebase Hosting (a global CDN).

Repository path: `v2/web_app/lab-access-nexus-main/`

9.2 Technology Stack

Table 9.1: Web Application Technology Stack

Layer	Technology	Purpose
Framework	React 18 + TypeScript	UI rendering and type safety
Build tool	Vite	Development server and production bundler
Styling	Tailwind CSS	Utility-first CSS
UI components	shadcn/ui	Pre-built accessible components
Backend	Firebase SDK v10	Firestore + RTDB + Auth client
Routing	React Router v6	Client-side navigation
State	React Context	Global auth and machine state
Hosting	Firebase Hosting	CDN-served static files

9.3 Authentication Flow

MMS V2 uses Firebase Authentication with email/password. After login:

1. Firebase Auth confirms credentials
2. Web app checks `/admins/{uid}` in Firestore
3. If document exists: user is an admin
4. Web app checks `/super_admins/{uid}` in Firestore
5. If document exists: user has super admin privileges

6. If neither document exists: user is logged in to Firebase but not an MMS admin — redirect to login with error

> The presence check in `/admins` and `/super_admins` is the authorisation gate. Firebase Auth alone is not sufficient — a user with a Firebase Auth account but no Firestore admin document cannot access the admin dashboard.

9.4 Feature Reference

Feature	Requires role	Implementation
Real-time machine status	Admin	RTDB `onValue` on `/mms/nodes/*`
Session log view	Admin	Firestore query on `/sessions`
User management (add/edit)	Admin	`setDoc` to `/users/{roll}`
Machine management	Admin	`updateDoc` on `/machines/{id}`
Remote stop session	Admin	Write to RTDB `/mms/commands/{id}`
Machine enable/disable	Admin	Update `machine_active` in Firestore
Card revocation	Admin	Create `/revoked/{uid}` document
OTA firmware update	Super Admin	Write to RTDB `/mms/ota`
Add/manage admins	Super Admin	Create `/admins/{uid}` document

Table 9.2: Web Application Feature Reference

9.5 Critical V1 Bugs Fixed in Web App

9.5.1 `addDoc` → `setDoc` for User Creation

V1 used `addDoc(collection(db, "users"), data)`, which creates documents with auto-generated IDs. This makes roll-number lookup impossible (requires full collection scan).

V2 uses:

```
// V1 (wrong): auto-generated ID
await addDoc(collection(db, "users"), userData);

// V2 (correct): roll number as document ID
await setDoc(doc(db, "users", rollNumber), userData);
```

Listing 9.1: User creation fix in src/lib/firebase.ts

9.5.2 PIN Hashing

V1 stored the PIN in plaintext. V2 hashes the PIN before storing:

```
export async function hashPin(pin: string): Promise<string> {
    const encoder = new TextEncoder();
    const data = encoder.encode(pin);
    const hash = await crypto.subtle.digest("SHA-256", data);
    return Array.from(new Uint8Array(hash))
        .map(b => b.toString(16).padStart(2, "0"))
        .join("")
        .substring(0, 16); // 8 bytes = 16 hex chars
}
```

Listing 9.2: PIN hashing in src/lib/firebase.ts

The full HMAC-SHA256 with master key and UID is computed by the kiosk during card write. The web app stores a simplified PIN hash used for kiosk login verification only.

9.6 Environment Configuration

All Firebase configuration is in the `.env` file (never committed to git):

```
VITE_FIREBASE_API_KEY=AIzaSy...
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=123456789
VITE_FIREBASE_APP_ID=1:123456789:web:abc123
VITE_FIREBASE_DATABASE_URL=https://your-project-default-rtdb.
   region.firebasedatabase.app
```

Listing 9.3: v2/web_app/lab-access-nexus-main/.env

All variables prefixed with `VITE_` are bundled into the production build by Vite and accessible in the browser. This is intentional — these are client-side Firebase config values, not secrets. Firebase security rules enforce authorisation.

9.7 Build and Deploy

```
cd v2/web_app/lab-access-nexus-main
npm install
npm run build          # outputs to dist/
firebase deploy --only hosting
```

The deployment URL is shown at the end of `firebase deploy`. Share this URL with lab admins.

Chapter 10

Security Model

10.1 Threat Model

MMS V2 is deployed in a university lab on a trusted LAN. The threat model assumes:

- **Insider threat (student)**: A student may attempt to use a machine they are not authorised for. A student may attempt to stay on a machine beyond the session limit.
- **Card theft or loss**: A student's RFID card may be stolen or lost. The attacker would attempt to use it to gain machine access.
- **Card forgery**: An attacker may attempt to create a fake RFID card with crafted block data.
- **Physical access to node**: An attacker may attempt to physically tamper with the ESP32 node.
- **WiFi LAN attacker**: An attacker on the same WiFi may attempt to inject MQTT messages.

Out of scope: Remote internet attackers (Firebase security rules handle this), state-level adversaries, physical break-ins.

10.2 Security Controls

10.2.1 Card Forgery Prevention

Each card's Sector 1 is protected by a unique 6-byte key derived as:

$$\text{sector\_key}[6] = \text{HMAC-SHA256}(\text{master\_key}, \text{uid\_hex})[0..5]$$

An attacker cannot read Sector 1 data without knowing the master key. An attacker cannot write to Sector 1 without the sector key. Since the sector key is derived from the card's own UID and the master key, creating a forged card with a new UID requires knowledge of the master key.

The MFRC522 **never returns Key A on a read** — even with authenticated access. So compromising one card does not reveal the sector key for another card.

10.2.2 Revocation

Lost or stolen cards are revoked via the web app. Revocation propagates to all nodes via MQTT within seconds. Revoked card UIDs are stored in LittleFS on each node, so revocation works even if the MQTT broker goes down (until the next reconnect, nodes use their cached revocation list).

Limitation: A revoked card that is used while a node is offline may be accepted until the node reconnects. This is a known trade-off in favour of offline availability.

10.2.3 Admin Web App

- Firebase Authentication protects admin login (email/password)
- Firestore security rules check `/admins/{uid}` and `/super_admins/{uid}` on every client request
- Sessions collection has `allow write: if false` — only the bridge (Admin SDK) can write sessions, preventing session forgery from the client
- Super admin privileges required for OTA and admin management

10.2.4 MQTT Security

In the default deployment, MQTT uses `allow_anonymous true`. This is acceptable in a physically secure lab LAN where all connected devices are managed. For hardened deployments:

- Enable MQTT password authentication (see Section 7.9)
- Add per-node ACLs to restrict which topics each node can publish/subscribe to
- Use TLS on MQTT (port 8883) if the lab network is untrusted

10.2.5 Secrets Management

Sensitive secrets are never committed to version control. The `.gitignore` covers:

- `v2/access_node/secrets.h` (WiFi credentials, master key)
- `v2/kiosk_station/station_writer/secrets_station.h` (master key)
- `v2/kiosk_station/secrets.py` (master key)
- `v2/rpi_bridge/service-account.json` (Firebase Admin SDK key)
- `v2/web_app/lab-access-nexus-main/.env` (Firebase client config)

10.2.6 Firmware Security

- OTA firmware is downloaded from Firebase Storage (HTTPS). `esp_https_ota()` validates the TLS certificate.
- The dual-partition OTA scheme with rollback prevents a bad firmware update from permanently bricking a node.

- The 30-second hardware watchdog ensures the node restarts in case of firmware hang, preventing a machine from being left in relay-ON state indefinitely.
- Relay GPIO is driven LOW before `pinMode(OUTPUT)`, ensuring the relay is OFF during boot regardless of the GPIO's default state.

10.3 Security Assumptions and Limitations

Table 10.1: Security Assumptions and Limitations

Assumption / Limitation	**Details**
Master key is secure	If the master key is stolen, all cards can be forged. The key must be stored in a password manager, not in plaintext on a shared drive.
Lab WiFi is trusted	MQTT uses no encryption. A packet sniffer on the same WiFi can read MQTT messages (but not master key or Firebase credentials, which are never in MQTT).
Revocation is not instant	Offline nodes cannot revoke cards immediately. A stolen card may work for minutes to hours if the affected node is offline.
Physical node security	An attacker with physical access to the ESP32 can read flash, extract master key. Epoxy potting of the PCB mitigates this.
PIN not verified at machine (V2)	Card tap alone is sufficient. A thief with a stolen card can use any permitted machine. Revocation is the mitigation.
Firebase free tier	Data stored in Google Cloud. Subject to Google's terms of service. For highly sensitive data, consider self-hosted alternatives.

10.4 Security Checklist for Future Maintainers

Before deploying any change, verify:

- `secrets.h`, `secrets.py`, `secrets_station.h` are NOT committed to git (`git status` should not show them)
- Firestore rules still have `allow write: if false` on the `sessions` collection
- RTDB rules protect `nodes` and `revoked` paths from unauthenticated writes
- New admin accounts are added to Firestore `/admins` or `/super_admins` only — do not grant access by adding to Firebase Auth alone
- OTA firmware URL uses HTTPS (Firebase Storage URLs always do)
- Master key backup is current and stored securely

Chapter 11

Deployment Guide

11.1 Deployment Overview

A complete MMS V2 deployment follows this order. Each step depends on the previous being complete.

Table 11.1: Deployment Sequence

Step	Task	Reference
1	Generate master key (ONCE)	Chapter 11, Section 11.2
2	Set up Firebase project	Deployment Package, Section 08
3	Set up Raspberry Pi	Deployment Package, Section 07
4	Deploy web application	Deployment Package, Section 09
5	Set up kiosk station	Deployment Package, Section 10
6	Assemble nodes	Chapter 4; Deployment Package, Section 05
7	Run hardware validation	Appendix J; Deployment Package, Section 06
8	Provision nodes	Deployment Package, Section 11
9	Run integration tests	Deployment Package, Section 12
10	Issue cards to students	Deployment Package, Section 10
11	Pilot deployment (1 machine)	Deployment Package, Section 13
12	Full rollout (all machines)	Deployment Package, Section 13

> The full deployment package is in `v2/deployment/`. This chapter summarises the most critical steps. Refer to the deployment package for step-by-step instructions with exact commands.

11.2 Master Key Generation

The master key is a 32-byte random secret shared across all nodes and the kiosk. It must be generated exactly once, before any card is issued or any node is provisioned.

```
python3 -c "
import secrets
bs = secrets.token_bytes(32)
print('C array:')
print('  ' + ', '.join(f'0x{b:02X}' for b in bs))
```

```
print()
print('Python hex:')
print('  ' + bs.hex())
"
```

Store the output in:

- A password manager (required)
- A printed copy in a sealed envelope in the lab safe (recommended)
- **Do not store in a shared Google Drive or any cloud storage in plaintext**

Use the C array form for `secrets.h` and `secrets_station.h`. Use the hex form for `secrets.py`.

> **Losing the master key means all existing cards must be replaced.** The key cannot be recovered from the cards or the firmware. If it is lost, generate a new key, re-flash all nodes, and re-issue all cards.

11.3 Node Provisioning Summary

For each node:

1. Copy `secrets.h.template` to `secrets.h`; fill in WiFi credentials and master key
2. Set `MACHINE_ID` and `MACHINE_NAME` in `config.h`
3. Flash firmware via Arduino IDE (Partition: Default 4MB with spiffs)
4. Verify boot log on Serial Monitor (115200 baud)
5. Confirm MQTT connect and config push from bridge
6. Test card access: permitted card GRANTED, non-permitted card DENIED
7. Complete provisioning checklist (Deployment Package, Section 11)

11.4 Critical Pre-Deployment Verification

Before any node goes to a live machine, run these checks:

```
# 1. Bridge healthy
curl http://192.168.0.10:8080/health
# Expect: {"mqtt_connected": true, "fb_connected": true, ...}

# 2. Mosquitto reachable
mosquitto_pub -h 192.168.0.10 -t mms/test -m ping
# Expect: no error

# 3. Firestore machines seeded
firebase firestore:get /machines/1
# Expect: JSON with "Creality 3D Printer"
```

```
12
13 # 4. Web app accessible
14 # Open dashboard URL in browser; login; confirm 5 machines
     shown
```

11.5 Secrets Files Reference

Table 11.2: Secret Files and Their Contents

File path	Contents
`v2/access_node/secrets.h`	WiFi SSID/password, MQTT broker IP, master key (C array)
`v2/kiosk_station/station_write_secrets.h`	Master key (C array)
`v2/kiosk_station/secrets.py`	Master key (Python hex bytes)
`v2/rpi_bridge/.env`	MQTT credentials, Firebase RTDB URL
`v2/rpi_bridge/service-account.json`	Firebase Admin SDK service account
`v2/web_app/lab-access-nexus-main/.env`	Firebase client config (not a secret per se, but not committed)

All of these are in `.gitignore`. Template files (e.g., `secrets.h.template`) are committed; actual secret files are not.

Chapter 12

Testing and Validation

12.1 Validation Strategy

MMS V2 validation is done at three levels:

1. **Hardware validation**: Each component on a node is tested in isolation with a dedicated Arduino sketch before production firmware is flashed.
2. **Integration testing**: All system components are tested together end-to-end after provisioning.
3. **Production verification**: After deploying to a live machine, basic access tests are repeated to confirm real-world behaviour.

12.2 Hardware Validation Suite

The hardware validation suite is in `v2/hardware_validation/`. It consists of 12 individual sketches plus one full-node integration sketch.

Table 12.1: Hardware Validation Sketches

#	Sketch	What it tests	Time
01	`01_nvs`	NVS read/write/clear	5 min
02	`02_rfid`	MFRC522 SPI, card UID read, auth	10 min
03	`03_relay`	Relay GPIO, continuity	5 min
04	`04_oled_spi`	128×64 SPI OLED display	5 min
05	`05_oled_i2c`	128×32 I2C OLED display	5 min
06	`06_hc89`	Slot sensor input, debounce	5 min
07	`07_ws2812`	WS2812 LED R/G/B cycle	5 min
08	`08_combined_display`	All displays simultaneously	10 min
09	`09_rotary_encoder`	EC11 CLK/DT/SW, interrupt	10 min
10	`10_wifi`	WiFi connect, NTP sync, reconnect	10 min
11	`11_mqtt`	MQTT QoS 0/1, retained, reconnect	15 min
12	`12_littlefs`	LittleFS mount/write/rotate/recover	10 min
99	`99_full_node`	All hardware simultaneously	10 min
		Total	**~2h 15m**

The recommended execution order minimises re-wiring: tests are grouped by wiring requirements. See `v2/hardware_validation/EXECUTION_PLAN.md` for the exact wiring diagrams, expected serial output, pass/fail criteria, and troubleshooting trees for each test.

12.3 Integration Test Suite

Integration tests verify the entire system working together. They are documented in the Deployment Package (Section 12) and briefly summarised here:

Table 12.2: Integration Test Summary

#	Test	Verifies
1	Normal session	Card tap → relay ON → RTDB updates → card remove → Firestore session logged
2	Access denial	Non-permitted card → relay stays OFF → no Firestore document
3	Session timeout	Card in slot past limit → relay turns OFF automatically → end_reason: timeout
4	Remote stop	Dashboard Stop button → relay OFF within 2s → end_reason: remote_stop
5	Card revocation	Revoke in web app → node denies within 5s
6	Offline session	RPi down → access still granted → session in LittleFS → reconnect → session in Firestore
7	Stale command TTL	Command posted when offline → reconnect after 6 min → command discarded
8	Config push	Change session limit in web app → node applies within 5s

A node passes integration testing if Tests 1–8 all pass.

12.4 Pass/Fail Criteria

Table 12.3: Integration Test Pass Criteria

#	Pass criterion	Notes
1	Relay ON <1s after card; session in Firestore with correct fields	Most important test
2	Relay never ON; zero Firestore documents	
3	Relay OFF at exact timeout; end_reason = “timeout”	Temporarily change limit to 1 min
4	Relay OFF within 2s of clicking Stop	Requires active session
5	Denial within 5s of web app revocation	
6	Session in Firestore after RPi restart	Must match content of LittleFS log
7	No relay action; command acknowledged in RTDB	Wait 6+ min after posting command
8	OLED shows new limit within 5s	

12.5 Regression Testing After Firmware Updates

After any firmware update (OTA or direct flash), run at minimum:

- Integration Test 1 (normal session)
- Integration Test 4 (remote stop)
- Integration Test 5 (revocation)

This takes approximately 15 minutes and confirms the core access control and command paths still work.

Chapter 13

Maintenance Procedures

13.1 Maintenance Schedule Summary

Table 13.1: Maintenance Schedule

Frequency	Task	What to check	Time
Daily	Dashboard health check	All machines showing status; no stuck active sessions	2 min
Weekly	RPi health check	Disk, RAM, bridge logs for errors	10 min
Weekly	Firestore index check	All 3 indexes show "Enabled"	2 min
Weekly	Card stock count	Reorder when <20 blank cards	2 min
Monthly	RPi SD card backup	`dd` image to secure storage	30 min
Monthly	Firestore quota review	Usage within Spark free tier limits	5 min
Monthly	Orphaned session cleanup	No `status:active` documents >3 hours old	10 min
Quarterly	MQTT password rotation	Change bridge and node MQTT passwords	15 min
As needed	Firmware update	OTA or direct flash; run regression tests	30 min

13.2 Firmware Update Procedure

1. Update `FW_VERSION` in `config.h` (e.g., "2.1.0")
2. Build in Arduino IDE: Sketch → Export Compiled Binary
3. Upload `.bin` to Firebase Storage: `firmware/2.1.0.bin`
4. Copy the download URL from the uploaded file
5. Write to RTDB `/mms/ota`:

```
{
  "firmware_url":   "https://firebasestorage.googleapis.com
   /.../2.1.0.bin",
  "target_version": "2.1.0",
  "released_at":    <current unix timestamp>
}
```

6. Monitor dashboard: each node shows "updating... → idle (v2.1.0)"
7. Verify on Serial Monitor: `[OTA] Update complete - v2.0.0 → v2.1.0`
8. Run regression tests (Chapter 12, Section 4)

> Always test OTA on one node at a bench before pushing to production. Keep the last known-good `.bin` file archived locally. If OTA fails on all nodes, manual USB flash recovery is required.

13.3 Card Management

13.3.1 Issuing a New Card

1. Admin creates student in web app (User Management → Add User)
2. At kiosk station: start Flask app (`python3 app.py`)
3. Select student from dropdown; confirm machine permissions
4. Click "Issue Card"; place blank MIFARE Classic 1K card on reader
5. Wait for "WRITE_SUCCESS"; card is personalised
6. Verify: take card to a node; confirm access is granted for permitted machines

13.3.2 Revoking a Card (Lost/Stolen)

1. Web app → User Management → select student → Revoke Card
2. All nodes receive revocation within 5s (if online)
3. Issue a replacement card (new blank card; new UID → new sector key)

13.4 Bridge Maintenance

```
# Check bridge status
sudo systemctl status mms-bridge

# Restart bridge (if needed)
sudo systemctl restart mms-bridge

# Watch logs live
sudo journalctl -u mms-bridge -f

# Check for errors in last 7 days
sudo journalctl -u mms-bridge --since "7 days ago" | grep -i "
    error|exception|failed"
```

13.5 Database Cleanup

13.5.1 Orphaned Sessions

Sessions with `status: "active"` older than 3 hours are orphaned (caused by power loss or network failure during session):

```
# On RPi, run the cleanup script
/home/pi/mms/venv/bin/python3 /home/pi/mms/cleanup_orphans.py
# Or add to crontab (runs daily at 3am):
# 0 3 * * * /home/pi/mms/venv/bin/python3 /home/pi/mms/
  cleanup_orphans.py
```

13.5.2 Stale RTDB Commands

The bridge auto-cleans commands older than 10 minutes. To clean manually:

- Firebase Console → RTDB → `mms/commands` → delete specific node's command

Chapter 14

Troubleshooting

14.1 Quick Diagnostic Reference

Table 14.1: Symptom to First Action

Symptom	First action
OLED blank	Check 3.3V on OLED VCC; flash test sketch 04 or 05
Card inserted, no reaction	Flash `06_hc89`; confirm HC89 OUT wired to GPIO 32
Card denied (should be allowed)	Check serial for denial reason; verify permissions in Firestore
Relay doesn't energise	Check relay LED; measure GPIO 26 with multimeter during session
MQTT keeps reconnecting	Check Mosquitto status on RPi; confirm MQTT_BROKER IP
Sessions missing from Firestore	Check bridge logs; curl bridge health endpoint
Dashboard shows offline	Check node MQTT; check bridge health; check RTDB subscription in web app
Remote stop not working	Check RTDB /mms/commands; check bridge logs; check 5-min TTL
OTA stuck	Check Firebase Storage URL; check bridge OTA listener in logs
Node rebooting in loop	Check serial for boot reason; watchdog or OTA crash rollback

14.2 Serial Monitor Diagnostic Commands

When a node is connected via USB, its Serial Monitor (115200 baud) provides the most direct diagnostic information:

Table 14.2: Diagnostic Log Prefixes

Prefix	Meaning
[HW]	Hardware initialisation events (boot only)
[NET]	WiFi connection events
[MQTT]	MQTT connection, publish, subscribe, receive events
[RFID]	Card detection and read events
[ACCESS]	Access grant or denial with reason
[RELAY]	Relay state changes
[SESSION]	Session start, end, timeout events
[FS]	LittleFS operations
[NVS]	NVS reads and writes
[OTA]	OTA update progress
[SYS]	System events: boot reason, watchdog, state transitions
[ERR]	Error conditions

14.3 Common Issues

14.3.1 RFID Authentication Failure

```
[RFID] Auth FAILED for UID: A1B2C3D4
```

Causes:

- Card written with a different master key than what is in `secrets.h`
- Card is not MIFARE Classic 1K (NFC tags, bank cards, transit cards fail)
- Card is a fresh/blank card with default Key A (`0xFFFFFFFFFFFF`) but the node tries the derived key first

Fix: Verify the master key is identical in all three locations. For fresh cards: the kiosk writes the derived key — node should then authenticate with the derived key. If a card was written by a different kiosk (different master key), it cannot be read by this node.

14.3.2 Schema Version Mismatch

```
[RFID] Schema version: 0x01 (expected 0x02) - ACCESS DENIED
```

Cause: Card was issued by V1 kiosk (old system).
Fix: Re-issue the card at the V2 kiosk.

14.3.3 Bridge PERMISSION_DENIED on Firestore

```
[FIREBASE] Error writing session: 403 PERMISSION_DENIED
```

Cause: The service account used by bridge.py does not have Firebase Admin SDK permissions.
Fix: Firebase Console → IAM → check the service account has "Firebase Admin" or "Editor" role. The service account email is shown in the `service-account.json` file.

14.3.4 WiFi RSSI Too Low

```
[NET] WiFi connected - IP: 192.168.0.101, RSSI: -84 dBm
```

Cause: Node is too far from the WiFi AP.

Impact: Intermittent MQTT disconnects; sessions buffered in LittleFS but may not flush promptly.

Fix: Add a WiFi access point or range extender nearer to the machine. Acceptable RSSI: > -80 dBm. Ideal: > -67 dBm.

14.4 Recovery Procedures

See the Deployment Package (Section 16) for complete disaster recovery procedures covering:

- Raspberry Pi hardware failure
- Firebase outage
- ESP32 hardware failure
- Lost master key
- Corrupted firmware (all nodes)
- Lost Firebase admin access
- Database corruption

Chapter 15

Future Improvements

15.1 Planned (Phase 4)

15.1.1 PIN Verification at Machine

The rotary encoder (EC11) is already wired (CLK=GPIO34, DT=GPIO35, SW=GPIO25). The card already stores the PIN hash (Block 5, bytes 0–7). What remains is the firmware implementation:

1. New firmware state: `PIN_ENTRY` (inserted between `CARD_READING` and `SESSION_ACTIVE`)
2. OLED UI: "Enter PIN: —-"; rotate encoder to select digit; SW to confirm
3. 30-second timeout for PIN entry
4. 3-attempt lockout before returning to IDLE
5. PIN hash verification: compute HMAC-SHA256(master_key‖uid_hex, entered_pin)[0..7] and compare with Block 5 bytes 0–7
6. No card re-issue required (PIN hash is already on the card)

15.2 Recommended (Near-term)

15.2.1 MQTT Authentication and ACLs

Enable `allow_anonymous false` in Mosquitto and add per-node ACLs. Without this, any device on the lab WiFi can publish to `mms/nodes/+/command` and trigger a remote stop or OTA on any node.

See deployment package Section 07 for the procedure.

15.2.2 Session Booking / Reservation

A booking system would allow students to reserve a machine time slot in advance. This would require:

- New Firestore collection: `/reservations`
- Web app booking UI (calendar view)
- Node integration: on card tap, check if current slot is reserved by this student (requires network call — breaks offline access)
- Design carefully for offline fallback (default to deny if no network, or default to allow)

15.2.3 Analytics Dashboard

The existing sessions collection contains all data needed for analytics. Adding a `/analytics` route to the web app would provide:

- Machine utilisation heat map (by hour and day of week)
- Per-student usage summary
- Peak demand periods
- Machines with highest timeout rate (students exceeding time limits)

No backend changes needed; all queries are on the existing `sessions` collection using the already-deployed Firestore indexes.

15.2.4 Card Expiry

Block 6 of Sector 1 is currently reserved (all zeros). It can be used to store an expiry date:

- Block 6 bytes 0–3: expiry Unix timestamp (uint32 LE)
- Block 6 byte 4: academic year
- Firmware adds one check: if `card.expiry_ts > 0 && now > expiry_ts` → deny with reason "expired"
- Cards would be renewed annually (re-issue with new expiry; permissions unchanged)

15.3 Long-term Considerations

15.3.1 Multi-Lab Deployment

The architecture supports multiple labs with the same Firebase project but separate RPis. Machine IDs must be globally unique across all labs. See deployment package Section 17 for the multi-lab design.

15.3.2 Replacing Firebase with Self-Hosted Backend

If the lab decides to move away from Google Firebase (data sovereignty, cost at scale, compliance):

- Firestore can be replaced with any document database (MongoDB, CouchDB, PostgreSQL with JSONB)
- RTDB can be replaced with Redis pub/sub or a second MQTT broker
- Firebase Hosting can be replaced with Nginx
- Firebase Auth can be replaced with Keycloak or Auth0

The bridge.py would need to be updated to use the alternative client libraries. The firmware does not connect to Firebase directly and would require no changes.

15.3.3 Hardware Iterations

- **Custom PCB**: Replace perfboard wiring with a custom PCB. Reduces assembly time from 4 hours to 30 minutes per node. Include all components in a single board form factor.
- **ESP32-S3**: The newer ESP32-S3 has more SRAM (512KB vs 320KB) and improved WiFi stability. Firmware changes required are minimal.
- **Ethernet instead of WiFi**: For machines in RF-shielded environments. ESP32 does not have built-in Ethernet; would require an ENC28J60 or W5500 Ethernet module.
- **Battery backup**: Add a small LiPo battery and charging circuit to maintain the ESP32 node during brief power outages. The relay would lose power anyway (machine is AC powered), but the node could log the power loss event and transmit it when AC returns.

15.4 Known Limitations to Address

Table 15.1: Known Limitations and Improvement Path

Limitation	Impact	Improvement
Revocation not instant (offline nodes)	Stolen card may work until node reconnects	Add Ethernet; ensure nodes are always online
PIN not verified at machine	Card theft = machine access	Phase 4 implementation
MQTT no TLS	Messages readable on LAN	Enable TLS on Mosquitto (port 8883)
No MQTT auth (default)	Any LAN device can inject commands	Enable after initial deployment
LittleFS 50KB buffer cap	∼500 sessions lost if RPi down >hours	Increase buffer (limited by flash partition)
Single RPi (SPOF)	No bridge = no cloud sync	RPi cluster (high availability Mosquitto)

Appendix A

GPIO Pin Mappings

A.1 Complete ESP32 GPIO Assignment Table

GPIO	Signal	Direction	Pull-up	Notes
0	Boot mode	IN	EXT	Must be HIGH on boot (do not connect to GND)
2	Onboard LED	OUT	None	WiFi status indicator; also used for boot mode
4	OLED64 RST	OUT	None	Active LOW reset; pulled HIGH by firmware after boot
5	MFRC522 SS (SDA)	OUT	None	SPI chip select for RFID reader
6–11	Flash memory	—	—	Connected to internal flash; DO NOT USE
12	Boot voltage select	IN	—	HIGH selects 1.8V flash voltage; LOW selects 3.3V; leave floating
13	Available	—	—	Not used in V2
14	Available	—	—	Not used in V2
15	Available	—	—	Not used in V2
16	OLED64 CS	OUT	None	SPI chip select for primary OLED
17	OLED64 DC	OUT	None	Data/Command select for primary OLED
18	SPI SCK	OUT	None	Shared SPI clock: RFID + OLED64
19	SPI MISO	IN	None	Shared SPI MISO: RFID only (OLED is write-only)
21	OLED32 SDA	BIDIR	Module 4.7kΩ	I2C data for secondary OLED
22	OLED32 SCL	OUT	Module 4.7kΩ	I2C clock for secondary OLED

GPIO	Signal	Direction	Pull-up	Notes
23	SPI MOSI	OUT	None	Shared SPI MOSI: RFID + OLED64
25	Encoder SW	IN	INPUT_PULLUP	Rotary encoder push button
26	Relay IN	OUT	None	Machine relay control; LOW before pinMode
27	MFRC522 RST	OUT	None	RFID module reset (active LOW)
32	HC89 OUT	IN	INPUT_PULLUP	Card slot sensor; LOW = card present
33	WS2812 DIN	OUT	None	RGB LED data; via 470Ω series resistor
34	Encoder CLK	IN	EXT 10kΩ to 3V3	Input-only pin; NO internal pull-up
35	Encoder DT	IN	EXT 10kΩ to 3V3	Input-only pin; NO internal pull-up
36	Available (input-only)	IN	EXT	Not used in V2
39	Available (input-only)	IN	EXT	Not used in V2

Table A.1: Complete GPIO Assignment Table for MMS V2 Access Node

Input-only GPIOs: Pins 34, 35, 36, and 39 are input-only on the ESP32. They cannot be configured as outputs and have no internal pull-up or pull-down resistors. Always provide external pull-up resistors (10kΩ to 3.3V) when using these pins with open-drain devices or floating signals.

A.2 SPI Bus Sharing

Both the MFRC522 and the SSD1306 SPI OLED share the same SPI bus:

Table A.2: SPI Bus Signal Assignment

Signal	GPIO	MFRC522	SSD1306 OLED64
SCK	18	SCK	D0
MOSI	23	MOSI	D1
MISO	19	MISO	(not connected)
CS	5	SDA	—
CS	16	—	CS

The SPI arbitration is handled by driving only one CS/SS pin LOW at a time. When communicating with the RFID reader, GPIO 5 is LOW and GPIO 16 is HIGH. When

communicating with the OLED, GPIO 16 is LOW. The Arduino SPI library handles this transparently when each device is initialised with its respective SS pin.

A.3 I2C Bus

The I2C bus (GPIO 21 SDA, GPIO 22 SCL) is used only for the 128×32 OLED. The SSD1306 module has onboard 4.7kΩ pull-up resistors to 3.3V on both lines. No external pull-ups are needed.

I2C address: **0x3C** (default for most SSD1306 128×32 modules). If you have a different module variant, scan with `i2cdetect` in a test sketch to confirm.

Appendix B

MQTT Topic Reference

B.1 Topic Naming Convention

All MMS V2 topics use the prefix `mms/`. The hierarchy is:

```
mms/
  nodes/
    {machine_id}/
      status      - node publishes live state
      event       - node publishes completed session events
      heartbeat   - node publishes health data
      command     - bridge publishes commands to node
      config      - bridge publishes config to node (retained)
  all/
    revoked       - bridge publishes revocation list (retained)
```

B.2 Topic Details

B.2.1 `mms/nodes/{id}/status`

- **Publisher**: ESP32 access node
- **QoS**: 1
- **Retained**: Yes
- **Last Will**: `{"state":"offline"}` (retained; broker publishes on unexpected disconnect)

```
{
  "state":            "active",
  "roll_number":      "220002123",
  "session_start":    1718600000,
  "firmware_version": "2.0.0",
  "ip_address":       "192.168.0.101",
  "last_seen":        1718600060,
  "rssi":             -55
}
```

Listing B.1: Status topic payload

Valid `state` values: `"offline"`, `"idle"`, `"card_present"`, `"active"`, `"error"`, `"ota"`.

B.2.2 mms/nodes/{id}/event

- **Publisher**: ESP32 access node
- **QoS**: 1
- **Retained**: No

```
{
  "t":  1718600900,
  "r":  "220002123",
  "m":  3,
  "e":  "session_end",
  "d":  245,
  "er": "card_removed"
}
```

Listing B.2: Event topic payload

Fields: `t`=end timestamp, `r`=roll number, `m`=machine_id, `e`=event type, `d`=duration_s, `er`=end_reason.

B.2.3 mms/nodes/{id}/heartbeat

- **Publisher**: ESP32 access node (every 10 seconds)
- **QoS**: 0
- **Retained**: No

```
{"ts": 1718600060, "heap_free": 218432, "uptime_s": 3600}
```

Listing B.3: Heartbeat topic payload

B.2.4 mms/nodes/{id}/command

- **Publisher**: RPi bridge (forwards from RTDB)
- **QoS**: 1
- **Retained**: No

```
{
  "cmd":       "stop",
  "issued_by": "admin_uid",
  "ts":        1718600100,
  "url":       null,
  "version":   null
}
```

Listing B.4: Command topic payload

Valid `cmd` values: `"stop"`, `"reboot"`, `"ota"`.

For OTA commands: `"url"` contains the firmware download URL, `"version"` contains the target version string.

B.2.5 mms/nodes/{id}/config

- **Publisher**: RPi bridge (from Firestore `onSnapshot`)
- **QoS**: 1
- **Retained**: Yes (nodes receive latest config on connect even if bridge is not running)

```
{
  "name":               "Soldering Station 1",
  "session_limit_min":  15,
  "relay_active_high":  true,
  "machine_active":     true
}
```

Listing B.5: Config topic payload

B.2.6 mms/all/revoked

- **Publisher**: RPi bridge (from Firestore `onSnapshot`)
- **QoS**: 1
- **Retained**: Yes (all nodes receive on connect; no need to re-push)

```
{
  "uids":       ["A1B2C3D4", "E5F60718"],
  "updated_at": 1718600300
}
```

Listing B.6: Revocation list topic payload

B.3 Subscribing to Topics for Debugging

```
# On the RPi or any machine with mosquitto-clients installed

# Watch all MMS traffic
mosquitto_sub -h 192.168.0.10 -t "mms/#" -v

# Watch only status from all nodes
mosquitto_sub -h 192.168.0.10 -t "mms/nodes/+/status" -v

# Watch events from node 3
mosquitto_sub -h 192.168.0.10 -t "mms/nodes/3/event" -v

# Watch revocation list
mosquitto_sub -h 192.168.0.10 -t "mms/all/revoked"

# Manually publish a stop command (for testing)
mosquitto_pub -h 192.168.0.10 -t "mms/nodes/3/command" \
  -m '{"cmd":"stop","issued_by":"test","ts":1718600100}'
```

Appendix C

Firestore Schema Reference

C.1 Collection Overview

Table C.1: Firestore Collections

Collection	Document ID	Purpose
`/users`	Roll number (9 digits)	Student profile, card UID, permissions, PIN hash
`/machines`	Machine ID (“1” to “5”)	Machine config: name, session limit, relay type, active state
`/sessions`	Auto-generated	Completed session records (permanent log)
`/revoked`	Card UID hex string	Revoked RFID card records
`/admins`	Firebase Auth UID	Admin user records
`/super_admins`	Firebase Auth UID	Super admin records

C.2 /users/{roll_number}

Field	Type	Description
`roll_number`	string	9-digit roll number; same as document ID
`name`	string	Student full name
`email`	string	Institute email address
`pin_hash`	string	HMAC-SHA256 PIN hash, truncated to 8 bytes (16 hex chars)
`card_uid`	string	RFID card UID as uppercase hex (e.g., “A1B2C3D4”)
`machine_permissions`	number	uint32 bitmask; bit N-1 = machine N access
`issued_at`	timestamp	When the card was issued
`issued_by`	string	Firebase Auth UID of the admin who issued the card
`is_active`	boolean	`false` = deactivated student (access denied even with valid card)
`last_seen`	timestamp	Set by bridge when a session ends for this student

Table C.2: /users collection fields

C.3 /machines/{machine_id}

Field	Type	Description
`name`	string	Display name (e.g., "Soldering Station 1")
`session_limit_minutes`	number	Maximum session duration in minutes
`relay_active_high`	boolean	`true` = GPIO HIGH energises relay; `false` = GPIO LOW
`machine_active`	boolean	`false` = machine admin-disabled; all access denied
`location`	string	Physical zone (e.g., "Zone C")
`created_at`	timestamp	When this document was created

Table C.3: /machines collection fields

C.4 /sessions/{auto_id}

Field	Type	Description
`machine_id`	string	Machine ID as string ("1" to "5")
`machine_name`	string	Machine name at time of session (denormalised for query convenience)
`roll_number`	string	Student roll number
`user_name`	string	Student name at time of session (denormalised)
`start_time`	timestamp	Session start time (UTC)
`end_time`	timestamp | null	Session end time; `null` for active sessions
`duration_s`	number | null	Session duration in seconds; `null` for active sessions
`end_reason`	string | null	Why session ended; see valid values below
`status`	string	"active" or "completed"

Table C.4: /sessions collection fields

Valid `end_reason` values:

- `card_removed`: student removed card from slot
- `timeout`: session exceeded limit; node ended session
- `remote_stop`: admin issued stop command via dashboard
- `power_loss`: node rebooted during active session (end time estimated)
- `watchdog`: 30s hardware watchdog triggered during session

C.5 /revoked/{card_uid}

Field	Type	Description
`uid`	string	Card UID hex string (same as document ID)
`roll_number`	string	Roll number of the student this card was issued to
`revoked_at`	timestamp	When the card was revoked
`revoked_by`	string	Firebase Auth UID of admin who revoked the card
`reason`	string	"lost", "stolen", or "expired"

Table C.5: /revoked collection fields

C.6 Firestore Composite Indexes

Three composite indexes are required on the `sessions` collection:

Table C.6: Required Firestore Composite Indexes

Collection	Fields	Purpose
sessions	machine_id ASC, start_time DESC	Per-machine session history
sessions	roll_number ASC, start_time DESC	Per-student session history
sessions	status ASC, start_time ASC	Active session queries and orphan cleanup

Deploy with: `firebase deploy -only firestore:indexes`

Appendix D

Realtime Database Schema Reference

D.1 Full RTDB Tree

```
{
  "mms": {
    "nodes": {
      "1": {
        "state":            "idle",
        "roll_number":      null,
        "session_start":    null,
        "firmware_version": "2.0.0",
        "ip_address":       "192.168.0.101",
        "last_seen":        1718600000,
        "rssi":             -52
      },
      "2": {
        "state":            "offline",
        "firmware_version": "2.0.0",
        "ip_address":       null,
        "last_seen":        1718500000,
        "rssi":             null
      },
      "3": {
        "state":            "active",
        "roll_number":      "220002123",
        "session_start":    1718600000,
        "firmware_version": "2.0.0",
        "ip_address":       "192.168.0.103",
        "last_seen":        1718600060,
        "rssi":             -67
      }
    },
    "commands": {
      "3": {
        "command":      "stop",
        "issued_by":    "admin_firebase_uid",
        "timestamp":    1718600100,
        "acknowledged": false
      }
    },
    "ota": {
      "firmware_url":   "https://firebasestorage.googleapis.com
   /.../firmware/2.1.0.bin",
```

```
      "target_version": "2.1.0",
      "released_at":    1718600200
    },
    "bridge": {
      "status": {
        "state": "online",
        "ts":    1718600000
      }
    }
  }
}
```

Listing D.1: Complete RTDB Structure

D.2 Node State Values

Table D.1: Valid values for `/mms/nodes/{id}/state`

State	Description
`"offline"`	Node is not connected to MQTT. Published by broker's LWT mechanism.
`"idle"`	Node is online, no card in slot.
`"card_present"`	Card detected in slot; RFID read in progress.
`"active"`	Session in progress; machine powered via relay.
`"error"`	Node encountered a hardware error and is in error recovery mode.
`"ota"`	Node is performing an OTA firmware update.

D.3 Command Lifecycle

1. Admin writes command to `/mms/commands/{id}`: `{command: "stop", acknowledged: false}`
2. Bridge forwards command to MQTT
3. Node receives, executes (or discards if stale)
4. Node publishes session event
5. Bridge or node sets `{acknowledged: true, ack_at: now}`
6. Bridge periodic sweep cleans up old acknowledged commands after 10 minutes

D.4 RTDB Security Rules

```
{
  "rules": {
    "mms": {
      "nodes": {
        ".read": "auth != null",
        ".write": false
      },
      "commands": {
        "$machine_id": {
          ".read": "auth != null",
          ".write": "auth != null"
        }
      },
      "ota": {
        ".read": "auth != null",
        ".write": "auth != null"
      },
      "bridge": {
        ".read": "auth != null",
        ".write": false
      }
    }
  }
}
```

Listing D.2: v2/database.rules.json

RTDB rules apply only to client-side SDK connections. The bridge.py uses the Firebase Admin SDK, which bypasses all RTDB security rules. This is by design: the bridge must be able to write node states, which clients cannot write directly.

Appendix E

Machine Registry

E.1 Canonical Machine Table

This table is the single source of truth for machine identity. Any discrepancy between this table and the code files is a bug.

Table E.1: Canonical Machine Registry

ID	Name	Session Limit	Perm Bit	Zone	Relay Type
1	Creality 3D Printer	60 min	Bit 0	Zone A	Mechanical
2	Fracktal Laser Cutter	30 min	Bit 1	Zone B	SSR
3	Soldering Station 1	15 min	Bit 2	Zone C	Mechanical
4	Soldering Station 2	15 min	Bit 3	Zone C	Mechanical
5	Bosche Grinder	20 min	Bit 4	Zone D	Mechanical

E.2 Three-Places-in-Sync Rule

Machine identity is specified in three code locations that must match exactly:

E.2.1 1. v2/access_node/config.h

```
// Canonical machine table (matches Firestore /machines/* documents)
// ID  Name                    Limit  Bit  Zone
// 1   Creality 3D Printer     60min  0    Zone A
// 2   Fracktal Laser Cutter   30min  1    Zone B
// 3   Soldering Station 1     15min  2    Zone C
// 4   Soldering Station 2     15min  3    Zone C
// 5   Bosche Grinder          20min  4    Zone D

#define MACHINE_ID    1
#define MACHINE_NAME  "Creality 3D Printer"
#define SESSION_LIMIT 60
#define MQTT_BROKER   "192.168.0.10"
```

Listing E.1: Machine identity in firmware config

E.2.2 2. v2/scripts/seed_machines.py

```
MACHINES = [
    {"id": "1", "name": "Creality 3D Printer",
     "session_limit_minutes": 60, "relay_active_high": True,
     "machine_active": True, "location": "Zone A"},
    {"id": "2", "name": "Fracktal Laser Cutter",
     "session_limit_minutes": 30, "relay_active_high": True,
     "machine_active": True, "location": "Zone B"},
    {"id": "3", "name": "Soldering Station 1",
     "session_limit_minutes": 15, "relay_active_high": True,
     "machine_active": True, "location": "Zone C"},
    {"id": "4", "name": "Soldering Station 2",
     "session_limit_minutes": 15, "relay_active_high": True,
     "machine_active": True, "location": "Zone C"},
    {"id": "5", "name": "Bosche Grinder",
     "session_limit_minutes": 20, "relay_active_high": True,
     "machine_active": True, "location": "Zone D"},
]
```

Listing E.2: Machine seeding script — source of truth for Firestore

E.2.3 3. v2/kiosk_station/config.py

```
# Canonical machine table -- must match Firestore /machines/*
   documents
MACHINE_IDS = {
    1: "Creality 3D Printer",
    2: "Fracktal Laser Cutter",
    3: "Soldering Station 1",
    4: "Soldering Station 2",
    5: "Bosche Grinder",
}
```

Listing E.3: Machine IDs in kiosk permission UI

E.3 Adding a New Machine

To add Machine 6 (example: Band Saw):

1. Add to the comment table in `config.h`: `// 6 Band Saw 25min 5 Zone E`
2. Add to `seed_machines.py` MACHINES list: `{"id": "6", "name": "Band Saw", ...}`
3. Add to `kiosk_station/config.py` MACHINE_IDS: `6: "Band Saw"`
4. Run `python3 scripts/seed_machines.py` to create the Firestore document
5. Assemble, validate, and provision a new node with `MACHINE_ID 6`

6. Update user permissions for students who need access to the new machine and re-issue their cards

Appendix F

Bill of Materials

F.1 Per-Node Components

The following table lists all components required for one access node.

#	Component	Qty	Approx. Cost	Notes
1	ESP32 NodeMCU 38-pin Dev Board	1	Rs.350	Must be 38-pin variant; 30-pin has different GPIO layout
2	MFRC522 RFID Reader Module	1	Rs.150	3.3V; comes with card and key fob
3	SSD1306 OLED 128×64 SPI	1	Rs.200	7-pin; white display recommended
4	SSD1306 OLED 128×32 I2C	1	Rs.150	4-pin; I2C address 0x3C
5	HC89 IR Slot Sensor	1	Rs.80	Slot width must accept MIFARE card thickness
6	5V Relay Module (single channel)	1	Rs.80	With optocoupler; for soldering station / grinder
7	SSR 25A (Solid State Relay)	1	Rs.350	For laser cutter; rated for inductive load
8	WS2812B RGB LED (5V)	1	Rs.30	Single LED; strip cut to 1 unit
9	HiLink HLK-PM01 (5V 1W)	1	Rs.220	AC-DC converter; 90-264V in, 5V/600mA out
10	470Ω resistor	1	Rs.2	For WS2812 data line; 1/4W
11	100µF 16V electrolytic capacitor	1	Rs.5	For WS2812 power supply filtering
12	10kΩ resistor	2	Rs.2	For encoder CLK and DT pull-up
13	EC11 Rotary Encoder	1	Rs.60	With push button; 20 detents per revolution
14	ABS project enclosure	1	Rs.200	Min 150mm × 100mm × 50mm
15	Cable gland PG9 (AC in)	1	Rs.20	For mains cable entry

#	Component	Qty	Approx. Cost	Notes
16	Cable gland PG7 (machine out)	1	Rs.20	For relay output cable
17	3-pin screw terminal blocks	3	Rs.30	For relay, HC89, power connections
18	AC fuse holder + 5A fuse	1	Rs.30	5A; on AC input before HiLink
19	3-core mains cable (1m)	1	Rs.40	For AC input to node
20	2-core wire (1m, 1mm^2)	1	Rs.25	For relay output to machine
21	Female dupont jumper cables (20cm)	1 set	Rs.50	For internal GPIO connections
22	USB cable (Type-A to Micro-B)	1	Rs.50	For programming; must be data cable
23	Perfboard (100mm × 80mm)	1	Rs.30	For mounting ESP32 and connectors
24	M3 standoffs + screws	1 set	Rs.30	For mounting boards inside enclosure
25	Heat shrink tubing (assorted)	1 set	Rs.30	For AC wire insulation
26	Electrical tape	1	Rs.30	Secondary insulation on AC connections
27	Solder + flux	—	Rs.50	For perfboard assembly
28	MIFARE Classic 1K cards	5	Rs.150	Blank; for initial testing and extras
	Per-node total (estimated)		**Rs.2,260**	Includes relay and SSR; actual cost depends on relay type used

Table F.1: Bill of Materials per Access Node

F.2 Infrastructure Components (One-time)

Table F.2: Infrastructure Bill of Materials

Component	Qty	Cost	Notes
Raspberry Pi 4B (2GB)	1	Rs.4,500	Or RPi 3B+; 1GB RAM minimum
Official RPi power supply (5.1V 3A)	1	Rs.800	NOT a phone charger
MicroSD card 32GB Class 10	1	Rs.500	Samsung Endurance Pro recommended
Ethernet cable (2m)	1	Rs.100	Cat5e or better

F.3 Kiosk Station Components

Table F.3: Kiosk Station Bill of Materials

Component	Qty	Cost	Notes
Laptop / PC	1	Existing	Any Linux/Mac/Windows with USB
ESP32 NodeMCU 38-pin	1	Rs.350	Dedicated to kiosk; runs station_writer.ino
MFRC522 RFID Reader Module	1	Rs.150	Dedicated to kiosk
USB cable	1	Rs.50	Laptop to ESP32
MIFARE Classic 1K cards (bulk)	100	Rs.3,000	Initial stock for all students

F.4 Total Cost Estimate

Table F.4: Total Deployment Cost Estimate (5 Nodes)

Item	Cost
5 access nodes (Rs.2,260 each)	Rs.11,300
Infrastructure (RPi + accessories)	Rs.5,900
Kiosk station	Rs.500
MIFARE cards (100 cards)	Rs.3,000
Spare components (10% buffer)	Rs.1,500
Total (approximate)	**Rs.22,200**

Costs are approximate as of 2024. Prices for ESP32 boards and components vary by supplier. Robu.in, Quartzcomponents, and Probots are reliable Indian suppliers. International suppliers (AliExpress) are cheaper but have longer lead times.

Appendix G

Wiring Diagrams

G.1 Full Node Wiring Diagram

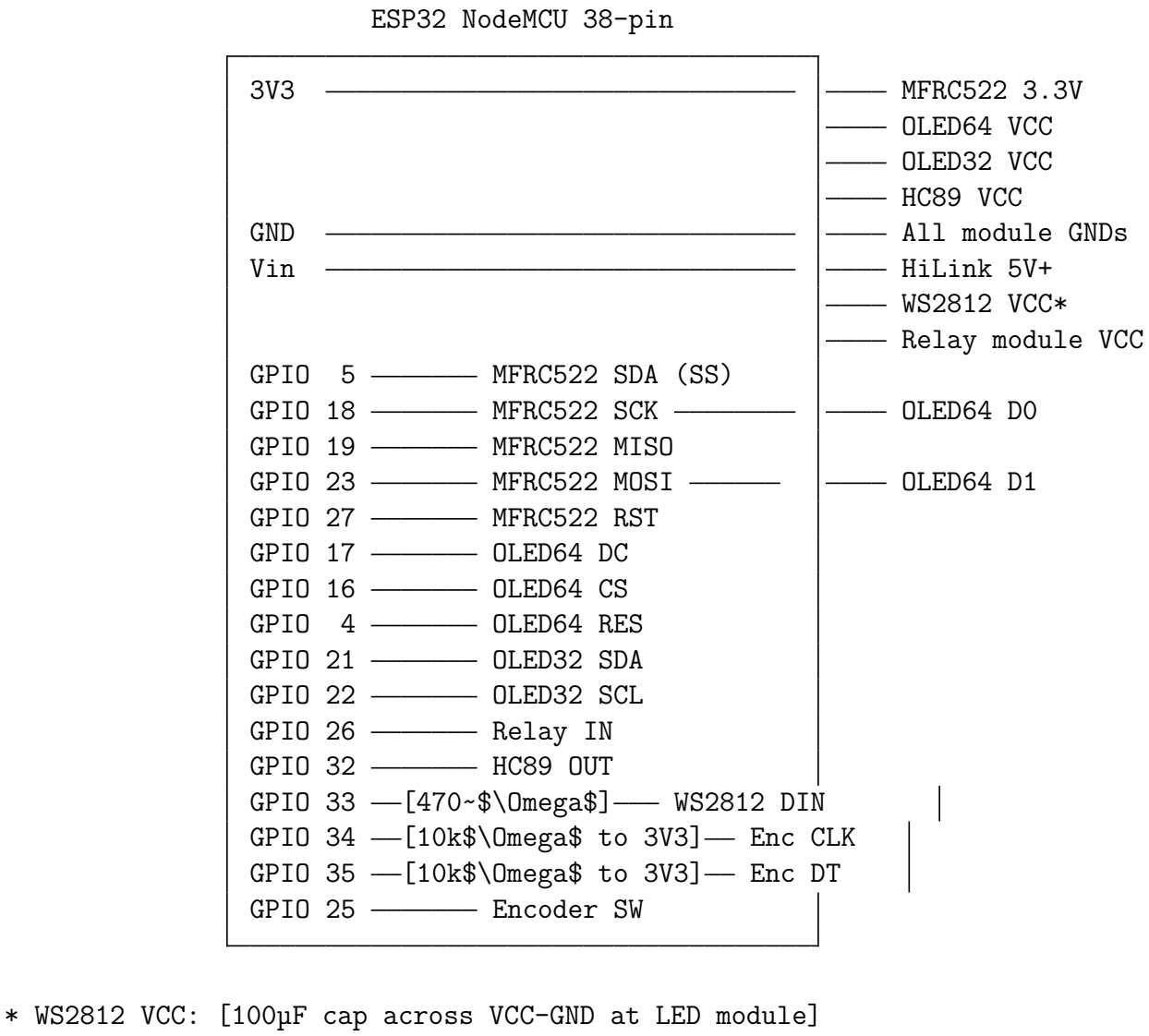


Figure G.1: Full Node Wiring Diagram

G.2 MFRC522 Connection Detail

```
MFRC522 module (8 pins, left to right):
  Pin 1: SDA  ──────── GPIO  5   (SPI chip select - NOT I2C SDA!)
  Pin 2: SCK  ──────── GPIO 18   (shared SPI clock)
  Pin 3: MOSI ──────── GPIO 23   (shared SPI MOSI)
  Pin 4: MISO ──────── GPIO 19   (shared SPI MISO)
  Pin 5: IRQ  ──────── NC        (not connected)
  Pin 6: GND  ──────── GND
  Pin 7: RST  ──────── GPIO 27   (active LOW reset)
  Pin 8: 3.3V ──────── 3V3       (NOT 5V - module is 3.3V only)
```

Figure G.2: MFRC522 Connection Detail

G.3 SSD1306 128x64 SPI OLED Connection

```
OLED 128x64 SPI module (7 pins):
  Pin 1: GND ──────── GND
  Pin 2: VCC ──────── 3V3
  Pin 3: D0  ──────── GPIO 18   (shared SCK with MFRC522)
  Pin 4: D1  ──────── GPIO 23   (shared MOSI with MFRC522)
  Pin 5: RES ──────── GPIO  4   (reset; active LOW)
  Pin 6: DC  ──────── GPIO 17   (Data/Command select)
  Pin 7: CS  ──────── GPIO 16   (SPI chip select)
```

Figure G.3: SSD1306 128x64 SPI OLED Connection

G.4 SSD1306 128x32 I2C OLED Connection

```
OLED 128x32 I2C module (4 pins):
  GND ──────── GND
  VCC ──────── 3V3
  SCL ──────── GPIO 22
  SDA ──────── GPIO 21

  Note: Module has onboard 4.7k$\Omega$ pull-ups.
        No external pull-up resistors needed.
        I2C address: 0x3C
```

Figure G.4: SSD1306 128x32 I2C OLED Connection

G.5 WS2812B LED Connection

```
WS2812B single LED:

  GPIO 33 ──[470~$\Omega$]──── DIN (data in)
  Vin (5V) ──────┬──────── VCC
                 [+]
               [100µF]    <- electrolytic, observe polarity
                 [-]
  GND ───────────┴──────── GND

  The 470~$\Omega$ resistor protects against signal reflections on long wire runs.
  The 100µF capacitor must be placed AT the LED, not at the ESP32.
```

Figure G.5: WS2812B LED Connection

G.6 HC89 Slot Sensor Connection

```
HC89 (3 or 4 pin):
  VCC ─────── 3V3  (or 5V if module requires it; check datasheet)
  GND ─────── GND
  OUT ─────── GPIO 32  (configured as INPUT_PULLUP in firmware)

  Signal behaviour:
    Card in slot  -> OUT = LOW  (IR beam blocked)
    No card       -> OUT = HIGH (IR beam reaches sensor)
    Firmware debounce: 50ms minimum hold time before state change
```

Figure G.6: HC89 Slot Sensor Connection

G.7 EC11 Rotary Encoder Connection

```
EC11 Rotary Encoder:
  CLK —[10k$\Omega$]——— 3V3          GPIO 34 ——— CLK
  DT  —[10k$\Omega$]——— 3V3          GPIO 35 ——— DT
  SW  (push button, one side)   GPIO 25 ——— SW  (INPUT_PULLUP in firmware)
  SW  (push button, other side) GND
  GND ——— GND

  WARNING: GPIO 34 and 35 are INPUT-ONLY pins with NO internal pull-up.
           External 10k$\Omega$ resistors to 3.3V are REQUIRED.
           Without them, the encoder signal is undefined and will
           trigger false interrupts.
```

Figure G.7: EC11 Rotary Encoder Connection

G.8 Relay Module Connection

```
Mechanical Relay Module (5V, optocoupler):
  VCC  ——— Vin (5V)
  GND  ——— GND
  IN   ——— GPIO 26

  Load (screw terminals):
  COM  ——— AC live wire from fused mains input
  NO   ——— Machine AC live wire in
  NC   ——— Not connected (normally closed; machine would run when relay is OFF)

  relay_active_high = true (HIGH = relay energised = machine ON)

Solid-State Relay (SSR 25DA):
  Control IN+  ——— GPIO 26  (3.3V signal; SSR accepts 3-32V DC)
  Control IN-  ——— GND
  Load 1       ——— AC live wire in
  Load 2       ——— Machine AC live wire in

  relay_active_high = true (HIGH = SSR conducts = machine ON)
```

Figure G.8: Relay Module Connection

G.9 Power Distribution

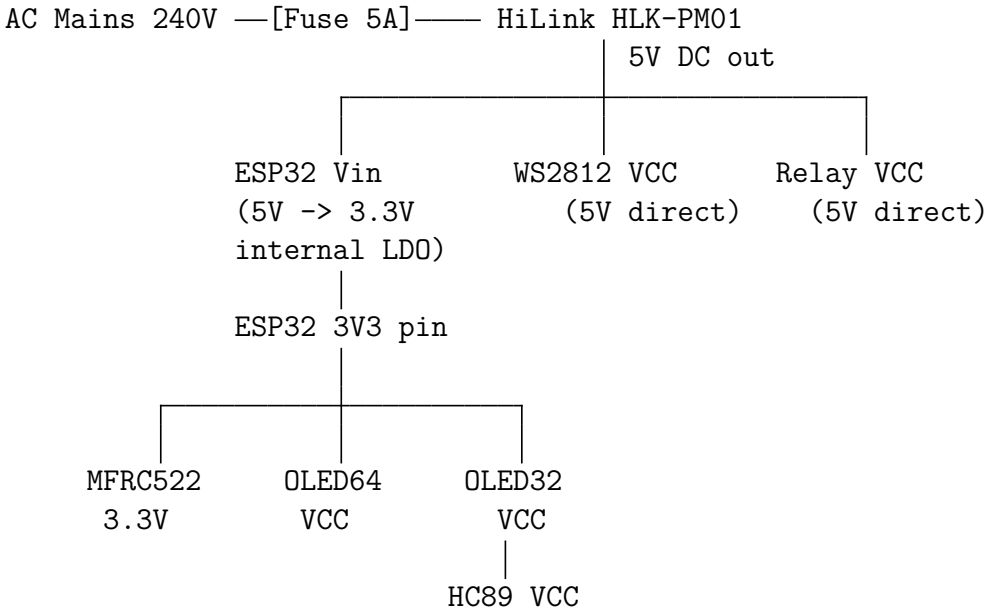


Figure G.9: Power Distribution Diagram

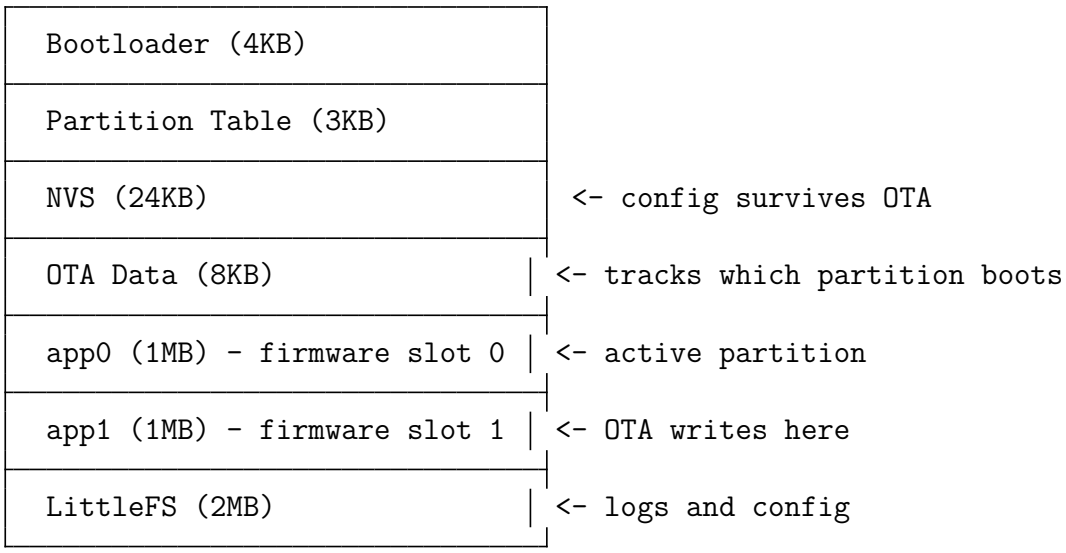
Appendix H

OTA Update Procedures

H.1 OTA Architecture

MMS V2 uses the ESP-IDF `esp_https_ota()` function for firmware updates. The dual-partition scheme allows automatic rollback if the new firmware is unstable.

ESP32 Flash (4MB):



H.2 OTA Update Procedure

H.2.1 Step 1: Build the Firmware

In Arduino IDE:

1. Open `v2/access_node/access_node.ino`
2. Update version: in `config.h`, change `#define FW_VERSION "2.0.0"` to the new version
3. Verify compile: Sketch → Verify/Compile (Ctrl+R)
4. Export binary: Sketch → Export Compiled Binary
5. The `.bin` file appears in the sketch directory

H.2.2 Step 2: Upload to Firebase Storage

1. Firebase Console → Storage → `firmware/` folder
2. Click Upload File, select the `.bin` file
3. Name it by version: `2.1.0.bin`
4. After upload: click the file → three-dot menu → Copy download URL
5. The URL is a long HTTPS URL; copy it exactly

H.2.3 Step 3: Trigger OTA

Option A — Via web app:

- Super Admin Panel → OTA Update → Paste URL and version → Submit

Option B — Via Firebase Console (RTDB):

1. Open RTDB → navigate to `/mms/ota`
2. Set the value to:

```
{
  "firmware_url":   "https://firebasestorage.googleapis.com
   /.../2.1.0.bin",
  "target_version": "2.1.0",
  "released_at":    1718600000
}
```

Option C — Via Firebase CLI:

```
firebase database:update /mms/ota --data '{
  "firmware_url":   "https://...",
  "target_version": "2.1.0",
  "released_at":    1718600000
}'
```

H.2.4 Step 4: Monitor Progress

On the dashboard: each node card shows "updating..." during OTA and reverts to "idle" with the new version number on success.

On Serial Monitor (if USB connected):

```
[OTA] Received OTA command: v2.0.0 -> v2.1.0
[OTA] URL: https://firebasestorage.googleapis.com/...
[RELAY] OFF (safety before OTA)
[OTA] Starting download...
[OTA] Progress: 10%
[OTA] Progress: 25%
...
[OTA] Progress: 100%
```

```
[OTA] Verification OK
[OTA] Rebooting to new firmware...
--- REBOOT ---
MMS V2 Access Node
FW: 2.1.0
...
[NVS] fw_version updated: 2.1.0
[MQTT] Published status: {fw_version: "2.1.0", ...}
```

H.3 OTA Rollback

The firmware includes automatic rollback:

- If the new firmware crashes 3 times on boot, the bootloader reverts to the previous partition
- The node reports the old firmware version on reconnect
- Manual rollback is not possible via OTA; re-trigger OTA with the previous known-good `.bin` file

H.4 Manual Flash Recovery

If OTA fails catastrophically (rollback also fails):

1. Connect USB cable from laptop to ESP32
2. Open Arduino IDE with the last known-good sketch
3. Upload via USB (*not* OTA)
4. After successful upload: node boots from the new firmware on app0 partition
5. Re-run integration tests before returning node to service

H.5 OTA Safety Checklist

Before triggering OTA on production nodes:

- [ ] New firmware tested on bench node (direct USB flash, then integration tests 1–8)
- [ ] OTA tested on one bench node before pushing to all production nodes
- [ ] Last known-good `.bin` file archived locally
- [ ] Relay turns OFF before OTA starts (safety: machines don't stay powered during update)
- [ ] RTDB OTA `released_at` timestamp is current Unix time (not a stale value)
- [ ] All nodes showing "idle" before OTA (no active sessions)

Appendix I

Card Issuance Procedures

I.1 Overview

Every student who uses lab machines needs a personalised RFID card. This appendix provides the complete card issuance reference for kiosk operators.

I.2 Required Setup

Before issuing any cards, confirm:

- station_writer.ino is flashed with correct master key
- Flask kiosk app is running: `python3 app.py`
- Firebase Firestore is reachable (internet connection)
- Blank MIFARE Classic 1K cards are available
- A test node is available to verify issued cards

I.3 Issuing a New Card

1. If student not yet in Firestore: go to web app → User Management → Add User. Fill in roll number, name, email, PIN, machine permissions. Click Save.
2. Open kiosk: `http://localhost:5000`
3. Select the student's roll number from the dropdown
4. Verify the machine permissions shown match what was set in the web app
5. Click "Issue Card"
6. App shows: "AWAITING_CARD — place card on reader"
7. Place one blank MIFARE Classic 1K card face-down on the MFRC522 reader
8. Wait for LED/display confirmation on the kiosk
9. App shows: "WRITE_SUCCESS"
10. Kiosk updates Firestore: `/users/{roll}` gets `card_uid` and `issued_at`
11. **Verify**: take card to a test node. Insert card. OLED should show the student's roll number and "Access Granted" for permitted machines.
12. Hand card to student with instructions on use

I.4 Re-issuing a Lost Card

1. Web app → User Management → select student → Revoke Card
2. Confirm revocation reason: "lost"
3. Wait 10 seconds for revocation to propagate to all nodes
4. Issue a new card following the new card procedure above
5. The new card has a new UID and therefore a new sector key — the old card cannot be used even if found

I.5 Re-issuing a Damaged Card (Not Lost)

1. NO revocation needed (physical card is destroyed; cannot be used by anyone)
2. Issue a new card following the new card procedure
3. The old UID may remain in Firestore `card_uid` field until the new card is issued, at which point it is overwritten

I.6 Updating Machine Permissions

If a student needs access to additional machines (or has access removed):

1. Web app → User Management → select student → Edit Permissions
2. Update the machine checkboxes
3. Click Save (Firestore is updated immediately)
4. **A new card must be issued** — the permission bitmask is stored on the physical card; Firestore update alone does not update existing cards
5. Issue a new card following the new card procedure (old card becomes invalid for the new permissions, though it still works with old permissions until revoked)
6. If old card should no longer work at all: revoke it first, then issue new card

I.7 Bulk Card Issuance

For issuing cards to a new batch of students at semester start:

1. Export student list from institute system (roll number, name, email)
2. Use batch user creation script (if available) or create users individually in web app
3. Assign machine permissions per student role (lab regular, advanced user, etc.)
4. Set up kiosk station in a central location

5. Call students in batches of 10–15
6. Issue cards assembly-line style: 1–2 minutes per card
7. Verify every 5th card at a test node (spot-check)

   Expected throughput: 20–30 cards per hour with one kiosk operator.

I.8 Card Stock Management

- Keep minimum 20 blank cards in stock at all times
- Reorder from Robu.in or similar: MIFARE Classic 1K 13.56MHz cards
- Bulk order: 100 cards recommended for initial deployment
- Estimated annual replacement rate: 10–20% of active students (loss and graduation)
- Store blank cards away from strong magnets and direct sunlight

I.9 Troubleshooting Card Issuance

Table I.1: Card Issuance Troubleshooting

Problem	**Likely Cause**	**Fix**
WRITE_FAILED	Card moved during write	Hold card still; retry
WRITE_FAILED	Card is not MIFARE Classic 1K	Use correct card type
WRITE_FAILED	Previous different master key	Discard card; use fresh card
AWAITING_CARD timeout	Card not detected	Check HC89/MFRC522 connection; try different card
WRITE_SUCCESS but node denies	Different master key in node vs kiosk	Verify both use same 32-byte key
App crash on start	secrets.py not filled	Fill `MASTER_KEY` in secrets.py

Appendix J

Hardware Validation Procedures

J.1 Overview

This appendix is the quick-reference version of the hardware validation procedures. For full wiring diagrams, expected serial output, and troubleshooting trees, see `v2/hardware_validation/EXECUTIO`

J.2 Required Tools and Setup

- Laptop with Arduino IDE 2.x installed
- ESP32 board package: *esp32 by Espressif*, version 3.1.x
- USB cable (data cable, not charge-only)
- Multimeter (DC voltage + continuity)
- All components from the BOM (Appendix F)

Required libraries (install via Arduino Library Manager):

Table J.1: Required Arduino Libraries

Library	Version
Adafruit SSD1306	2.5.9
Adafruit GFX Library	1.11.9
MFRC522	1.4.11
Adafruit NeoPixel	1.12.3
PubSubClient	2.8.0
ArduinoJson	7.3.1

Board settings (same for every sketch):

- Board: ESP32 Dev Module
- Partition Scheme: Default 4MB with spiffs
- Upload Speed: 921600
- CPU Frequency: 240MHz
- Serial Monitor: 115200 baud

J.3 Execution Order

Table J.2: Recommended Test Execution Order

Order	Test	What to wire	Pass condition	Time
1	01 NVS	Nothing (USB only)	"PASS" on all data types	5 min
2	12 LittleFS	Nothing	All 7 phases PASS	10 min
3	10 WiFi	Nothing (built-in antenna)	IP assigned; NTP synced IST	10 min
4	11 MQTT	Nothing (needs RPi)	QoS 0 and QoS 1 PASS	15 min
5	05 I2C OLED	OLED32 only	Text visible on display	5 min
6	04 SPI OLED	OLED64 + OLED32	Text visible on OLED64	5 min
7	02 RFID	OLED64 + MFRC522	Card UID read correctly	10 min
8	03 Relay	Relay only	Multimeter: continuity on NO	5 min
9	06 HC89	HC89 slot sensor	PRESENT/ABSENT toggle reliably	5 min
10	07 WS2812	WS2812 LED	R/G/B cycle visible	5 min
11	08 Combined	All displays + LED	All active simultaneously	10 min
12	09 Encoder	Encoder + 2× 10kΩ	CW/CCW/button all detected	10 min
13	99 Full node	Everything connected	All components PASS	10 min

J.4 Sign-off Sheet

Test	Sketch	PASS	FAIL
NVS	01_nvs	☐	☐
LittleFS	12_littlefs	☐	☐
WiFi	10_wifi	☐	☐
MQTT	11_mqtt	☐	☐
I2C OLED 32px	05_oled_i2c	☐	☐
SPI OLED 64px	04_oled_spi	☐	☐
RFID Reader	02_rfid	☐	☐
Relay	03_relay	☐	☐
HC89 Slot Sensor	06_hc89	☐	☐
WS2812 LED	07_ws2812	☐	☐
Combined Display	08_combined_display	☐	☐
Rotary Encoder	09_rotary_encoder	☐	☐
Full Node	99_full_node	☐	☐
Machine ID:			
Date:			
Engineer:			
Sign-off:			

Condition for proceeding to provisioning: All 13 tests must show PASS. Any FAIL must be resolved and re-tested before the node proceeds to node provisioning (Appendix J, deployment package Section 11).

J.5 Common Failure Modes

Test	Failure	Most likely cause
NVS	Write fails	Flash issue (erase flash: Tools → Erase Flash)
LittleFS	Mount fails	Wrong partition scheme; must be "Default 4MB with spiffs"
WiFi	Connection fails	Wrong SSID/password; 5GHz AP (ESP32 = 2.4GHz only)
MQTT	Connection fails	RPi not running; wrong MQTT_BROKER IP; firewall
I2C OLED	Init fails	Wrong I2C address (try 0x3D if 0x3C fails); SDA/SCL swapped
SPI OLED	Init fails	CS or DC pin wired wrong; check GPIO 16/17
RFID	No card read	MFRC522 on 5V (must be 3.3V); RST not wired
Relay	No toggle	GPIO 26 wired to wrong pin; relay_active_high polarity

Test	Failure	Most likely cause
HC89	Always PRESENT	OUT not wired to GPIO 32; GPIO floating
WS2812	Wrong colour	Colour order; try NEO_RGB if NEO_GRB gives wrong colours
Encoder	No rotation detect	Missing 10kΩ pull-up on CLK/DT (GPIO 34/35 have none)

Table J.3: Hardware Validation Common Failures